

A Reliability-Graded Evaluation of Set Transformer Interaction Modeling for Scent Prediction in Two-Molecule Fragrance Blends

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Ian Peter L. Lastimosa

Jelly P. Aureus, MS

Jerome G. Flores

Jasper V. Gomez

September 2026

Naga City, Philippines

Keywords: graph neural networks, multi-label classification, set learning

Copyright 2026, Sofia Gabrielle I. Belarmino and Lester Liby V. Ojeñar

The Senior Project entitled

**A Reliability-Graded Evaluation of Set Transformer Interaction
Modeling for Scent Prediction in Two-Molecule Fragrance Blends**

developed by

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees
has been rigorously examined and recommended for approval and acceptance.

Jelly P. Aureus, MS

Panel Member

Date signed: _____

Jerome G. Flores

Panel Member

Date signed: _____

Jasper V. Gomez

Panel Member

Date signed: _____

Ian Peter L. Lastimosa

Project Advisor

Date signed: _____

The Senior Project entitled

**A Reliability-Graded Evaluation of Set Transformer Interaction
Modeling for Scent Prediction in Two-Molecule Fragrance Blends**

developed by

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Jelly P. Aureus, MS

Chair, Department of Computer Science

Date signed: _____

Marianne A. Tolentino, MS

Dean, College of Computer Studies

Date signed: _____

Declaration of Original Work

We declare that the Senior Project entitled

A Reliability-Graded Evaluation of Set Transformer Interaction Modeling for Scent Prediction in Two-Molecule Fragrance Blends

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

Sofia Gabrielle I. Belarmino

4 - Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Ian Peter L. Lastimosa

Project Advisor

A Reliability-Graded Evaluation of Set Transformer Interaction Modeling for Scent Prediction in Two-Molecule Fragrance Blends

by

Sofia Gabrielle I. Belarmino and Lester Liby V. Ojeñar

Project Advisor: Ian Peter L. Lastimosa

Department of Computer Science

ABSTRACT

Combining two molecular representations is an unordered-pair learning problem: exchanging the two inputs should not change the prediction, while the model may still need to preserve relationships between them. This study will compare mean pooling, order-averaged concatenation, and Set Transformer aggregation for multi-label prediction of catalog-recorded descriptors of two-molecule blends. The three variants will use a shared molecular graph encoder and matched training conditions. Order-averaged concatenation will be treated as an adapted baseline based on a published pairwise concatenation design, not as a full reproduction of that model.

The catalog descriptors will be treated as proxy targets rather than controlled sensory-panel measurements. Reliability grading and component-disjoint partitioning will be used as evaluation controls: the first will account for uncertain catalog annotations, while the second will reduce repeated molecular components across data partitions. The models will be compared on reliability-screened test cells using macro F1 as the primary metric and micro F1, PR-AUC, and Hamming loss as supporting metrics. Additional analyses will examine catalog-label patterns and how the relative performance of the three methods changes when training-label reliability is reduced, including under controlled label corruption. Numerical results will be reported only after the finalized evaluation pipeline has been run.

TABLE OF CONTENTS

1	Introduction	1
1.1	Project Context	2
1.2	Purpose and Description	3
1.3	Objectives	5
1.4	Scope and Limitations	5
1.5	Definition of Terms	7
2	Methodology	9
2.1	Research Design	9
2.2	Data Acquisition and Preparation	10
2.2.1	Data Sources and Provenance	11
2.2.2	Ground Truth and Its Limits	12
2.2.3	Target Vocabulary	12
2.2.4	Label Families (Reporting Only)	13
2.3	Label Reliability Mask	14
2.3.1	Consistency Evidence	14
2.3.2	Eligibility Mask	14
2.3.3	Experimental Label Conditions	15
2.4	Model Architecture	16
2.4.1	Shared Graph Encoder	16
2.4.2	Aggregation Variants	17
2.4.3	Output Head and Loss	19

2.5	Component-Disjoint Partitioning and Graph Carving	19
2.5.1	Scaffold Split Decay	19
2.5.2	Cluster-Based Partitions	20
2.5.3	The Graph-Carving Protocol	21
2.5.4	Training-Only Vocabulary Filter	21
2.6	Experiments	22
2.6.1	Experiments A and B: Main Comparison (RQ1)	22
2.6.2	Experiment C: Controlled Label Corruption (RQ3)	22
2.6.3	Blend-Type Analysis (RQ2)	23
2.7	Evaluation Metrics	23
2.7.1	Multi-Label Metrics	23
2.7.2	Masked Metrics	24
2.8	Statistical Comparison	25
2.9	Reporting Conventions	26
2.9.1	The Principal Odor Map	27
2.9.2	Limitations of Single-Molecule Models	28
2.10	Psychophysics of Odor Mixtures	28
2.10.1	Scent Blending Mechanics	28
2.10.2	Evidence That Averaging May Be Sufficient	29
2.11	Machine Learning on Sets and Chemical Mixtures	30
2.11.1	Permutation Invariance on Sets	30
2.11.2	Precedents in Mixture Modeling	31
2.12	Olfactory Datasets and Label Reliability	33
2.12.1	Reference Catalogs	33
2.12.2	The Blended-Pair Dataset	33
2.12.3	Human-Rated Datasets	34
2.12.4	Absence of a Standard Vocabulary	34
2.12.5	Noisy and Imbalanced Labels in Multi-Label Learning	34
2.13	Out-of-Distribution Splitting Strategies	35
2.14	Evaluating and Comparing Models Under Label Sparsity	36
2.15	Synthesis and Research Gap	37

2.15.1	Synthesis of Literature	37
2.15.2	Statement of the Research Gap	38
2.15.3	Positioning of the Present Study	38
3	Theoretical and Conceptual Framework	39
3.1	Theoretical Framework	39
3.1.1	Representing Molecules as Graphs	39
3.1.2	Graph Neural Networks and Message Passing	40
3.1.3	Graph Attention Encoder	41
3.1.4	Combining an Unordered Pair	43
3.1.5	Representation Trade-offs	45
3.1.6	Sparse Labels and Label Reliability	45
3.2	Conceptual Framework	47
3.2.1	Input Phase	47
3.2.2	Process Phase 1: Data Preparation and Graph Carving	48
3.2.3	Process Phase 2: Encoding and Pair Representation	48
3.2.4	Process Phase 3: Classification and Optimization	49
3.2.5	Output Phase	50
3.3	Technical Background	50
3.3.1	Cheminformatics Standard (RDKit): SMILES Parsing and Sanitization . . .	50
3.3.2	Deep Learning Stack: Sparse Graph Convolutions	51
3.3.3	Multi-Label Formulation: Sigmoids and Sparsity Mitigation	52
3.3.4	Evaluation Metrics for Sparse Multi-Label Outputs	53
4	Methodology	54
4.1	Research Design	54
4.2	Data Acquisition and Preparation	55
4.2.1	Data Sources and Provenance	56
4.2.2	Ground Truth and Its Limits	57
4.2.3	Target Vocabulary	57
4.2.4	Label Families (Reporting Only)	58
4.3	Label Reliability Mask	59

4.3.1	Consistency Evidence	59
4.3.2	Eligibility Mask	59
4.3.3	Experimental Label Conditions	60
4.4	Model Architecture	61
4.4.1	Shared Graph Encoder	61
4.4.2	Aggregation Variants	62
4.4.3	Output Head and Loss	64
4.5	Component-Disjoint Partitioning and Graph Carving	64
4.5.1	Scaffold Split Decay	64
4.5.2	Cluster-Based Partitions	65
4.5.3	The Graph-Carving Protocol	66
4.5.4	Training-Only Vocabulary Filter	66
4.6	Experiments	67
4.6.1	Experiments A and B: Main Comparison (RQ1)	67
4.6.2	Experiment C: Controlled Label Corruption (RQ3)	67
4.6.3	Blend-Type Analysis (RQ2)	68
4.7	Evaluation Metrics	68
4.7.1	Multi-Label Metrics	68
4.7.2	Masked Metrics	69
4.8	Statistical Comparison	70
4.9	Reporting Conventions	71
A	Chemical Feature Vectors	72
A.1	Symmetry Collapse: Representation-Theoretic Derivation	74
A.2	Dataset Profiles and Canonical Vocabulary	75
A.2.1	Single-Molecule Harmonized Dataset Sample	75
A.2.2	Pairwise Blend Mixture Sample	75
A.2.3	Standardized 104-Odor Descriptor Vocabulary Gate	75

LIST OF FIGURES

3.1 Input-Process-Output (IPO) Model. 47

LIST OF TABLES

2.1	Varied factors and matched controls.	10
2.2	Eligibility of a blend–descriptor cell in the Gold-only treatment.	15
2.3	The three aggregation variants.	17
3.1	Aggregation methods for an unordered pair of molecule embeddings.	43
4.1	Varied factors and matched controls.	55
4.2	Eligibility of a blend–descriptor cell in the Gold-only treatment.	60
4.3	The three aggregation variants.	62
A.1	Specification of the 46-Dimensional Atom (Node) Feature Vector (X_i).	73
A.2	Specification of the 10-Dimensional Bond (Edge) Feature Vector (E_{ij}).	73
A.3	Representative sample of harmonized single odorants.	75
A.4	Sample pairwise odor mixtures from the Sisson et al. benchmark with emergent descriptors.	75
A.5	The 104 standardized odor descriptors (alphabetical).	76

Chapter 1

Introduction

Combining two graph-derived vectors is an unordered-pair learning problem. Each molecule can be encoded as a graph and represented by a learned vector, but a blend should produce the same prediction whether the inputs are presented as (A, B) or (B, A) . The combination method must therefore preserve input-order invariance while retaining information about the particular pair being modeled. Mean pooling is a simple order-independent baseline, but it can map different pairs to the same representation: the pairs $(2, 8)$ and $(5, 5)$ have the same average even though their elements differ. More expressive aggregation may preserve distinctions that averaging loses, but it also adds parameters and computation. Whether that added complexity produces a useful improvement is an empirical question, not an assumption of this study [49].

Fragrance-blend prediction provides the application setting for this pairwise learning problem. Graph Neural Networks (GNNs) have been used to learn molecular representations directly from structure, including in single-molecule odor prediction [27, 34]. Extending a single-molecule encoder to a two-molecule input requires a separate aggregation design. This study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation for predicting catalog-recorded descriptors of binary fragrance blends [28, 44].

The blend descriptors used for training and evaluation are catalog records, not direct measurements of human perception. Public blend datasets such as the one compiled by Sisson et al. [44] are based on reference-catalog recommendations rather than a controlled sensory panel. The study therefore measures label consistency using reciprocal and cross-catalog agreement before comparing

the aggregation methods, and reports performance under declared reliability conditions rather than treating every catalog label as unquestioned ground truth.

1.1 Project Context

The fragrance domain supplies a useful application for studying this representation problem. Existing molecular GNNs can encode individual structures, but a pairwise blend task requires a second design decision: how to combine two molecule embeddings into one order-independent representation before multi-label classification [27, 44]. The unresolved issue addressed here is therefore not computational olfaction in general. It is the controlled comparison of alternative pair-combination methods under the same encoder, training conditions, and evaluation protocol.

The data structure also creates a partitioning problem. A blend dataset is organized as pair records, and the same molecule can occur in many different pairs. A random split of pair records can therefore place one molecule in both training and test partitions, allowing the model to reuse a component it has already seen. This study uses a component-disjoint split and graph-carving procedure to reduce that overlap by excluding pairs that cross the assigned partitions. The procedure is intended to reduce component leakage; it does not guarantee that every held-out molecule belongs to an entirely unfamiliar chemical family [13].

The labels create a second evaluation problem. Each blend may have several descriptors, but only a small fraction of the possible descriptor cells are positive. A model can therefore obtain a deceptively favorable aggregate score by predicting mostly absent labels, while inconsistent catalog records can change the apparent ranking of the aggregation methods. The study addresses these risks through multi-label metrics suited to sparse outputs and through a reliability audit based on reciprocal listing and cross-catalog agreement. The audit is used to define declared reliability conditions; it does not convert catalog records into verified measurements of human perception.

Fragrance and catalog terminology are retained only to identify the input domain and target descriptors. The study does not attempt to infer receptor mechanisms, explain physical mixture chemistry, or model perfume-formulation practice. Its computer-science focus is the representation and evaluation of an unordered pair of molecular graphs.

1.2 Purpose and Description

This study contributes a controlled empirical comparison of mean pooling, order-averaged concatenation, and Set Transformer aggregation under a shared molecular-graph encoding framework. It does not propose a new neural architecture. Instead, it evaluates whether an existing attention-based set aggregator provides a measurable benefit over simpler alternatives for predicting catalog-recorded descriptors of two-molecule blends [28, 44].

In all three variants, a Graph Attention Network (GAT) encodes each molecular graph into a learned vector using the same architectural specification and initialization policy [58]. The experimental variants differ in how the two molecule vectors are combined. Mean pooling averages the vectors directly. Order-averaged concatenation evaluates both input orders and averages their outputs. Set Transformer aggregation uses Self-Attention Blocks (SAB) and Pooling by Multihead Attention (PMA) to learn statistical relationships between the two vectors before producing an order-independent blend representation [28, 60].

A shared multi-label classification design maps each combined representation to independent sigmoid outputs for the target catalog descriptors. The variants are compared using matched data partitions, training conditions, and evaluation metrics. Reliability-aware evaluation reduces the influence of uncertain catalog annotations, while component-disjoint evaluation reduces the risk that the same molecular component appears in both training and test pairs. These controls strengthen the comparison, but they do not establish label correctness, eliminate all structural similarity, or guarantee that the Set Transformer will outperform the simpler methods.

Significance of the Study

The study is primarily intended to contribute evidence for computer-science research on pairwise representation learning and model evaluation. Its findings may be useful to the following groups:

- **Machine-Learning and Graph-Representation Researchers.** The study provides a controlled comparison of three methods for combining an unordered pair of graph embeddings. It contributes evidence on whether an attention-based set aggregator offers a measurable benefit over simpler methods, how model comparisons change under uncertain labels, and how component-disjoint splitting can reduce the influence of repeated molecular components across data partitions. Computational olfaction serves as the test domain for this broader represen-

tation and evaluation problem.

- **Researchers and Maintainers of Olfactory Reference Catalogs.** The reciprocal and cross-catalog agreement analyses provide a way to describe the consistency of catalog-derived labels before they are used for machine learning. These analyses may help identify descriptors that require closer review, although they do not determine whether a catalog label is perceptually correct.
- **Fragrance Researchers and Practitioners.** The results may provide preliminary evidence about the strengths and limits of structure-based prediction for catalog-recorded blend descriptors. However, this study does not validate a formulation tool or measure reductions in laboratory work, material use, or trial-and-error blending [38].

Research Questions

This study seeks to answer the following questions:

- RQ1.** Under matched encoding, training, and evaluation conditions, how does the performance of Set Transformer aggregation compare with mean pooling and order-averaged concatenation on the reliability-screened test data?
- RQ2.** How does the performance of Set Transformer aggregation relative to the two baselines vary across additive, catalog-label loss, and catalog-label gain patterns?
- RQ3.** How does the robustness of Set Transformer aggregation compare with the two baselines as training-label reliability decreases, including under controlled label corruption?

The study also has a limited connection to the United Nations Sustainable Development Goals. Its work on computational methods and data-quality evaluation is most closely related to SDG 9. Structure-based prediction could eventually support tools related to responsible material use under SDG 12, but the present study does not measure material savings, reduced laboratory work, or reduced trial-and-error formulation. These are possible future applications, not outcomes evaluated in this research.

1.3 Objectives

General Objective

To compare mean pooling, order-averaged concatenation, and Set Transformer aggregation as methods for combining an unordered pair of molecular graph embeddings for multi-label prediction of catalog-recorded blend descriptors, using matched training conditions and a reliability-aware, component-disjoint evaluation protocol.

To achieve the general objective, the following specific objectives must be accomplished:

- To prepare the single-molecule and blend datasets and describe the consistency and sparsity of their catalog-recorded labels.
- To construct fixed component-disjoint data partitions and reliability-screened evaluation conditions for comparing the models.
- To implement mean pooling, order-averaged concatenation, and Set Transformer aggregation using the same GAT encoder design, output task, loss function, and matched training protocol.
- To compare the three aggregation methods using macro F1 as the primary metric and micro F1, PR-AUC, and Hamming loss as supporting metrics.
- To determine how the relative performance of the three methods changes across label-reliability conditions, controlled label corruption, and additive, catalog-label loss, and catalog-label gain patterns.

1.4 Scope and Limitations

This study is limited to pairwise, two-molecule blends and the prediction of catalog-recorded descriptors. Each molecule is represented as a two-dimensional molecular graph and encoded by a shared Graph Attention Network (GAT) [58]. The study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation as three methods for combining the resulting molecule embeddings. The Set Transformer uses attention to learn statistical relationships between the embeddings; it is not treated as a model of receptor activity or physical molecular interaction [28, 60].

The data are obtained from the GoodScents and Leffingwell single-molecule catalogs and the catalog-derived binary-blend dataset of Sisson et al. [7, 44]. The target descriptors are binary catalog records rather than intensity ratings or measurements from a newly conducted sensory panel. The study does not recruit human assessors and does not claim that its predictions represent what every person would perceive. Concentration, mixing ratio, and changes over time are also outside the scope; each blend is treated as an unweighted pair.

Multi-label learning requires both positive and negative targets. A descriptor recorded for a blend is treated as positive, while an unlisted descriptor is treated as negative under a closed-world assumption. This is an operational definition for model training and evaluation. Catalog omission may reflect an unrecorded observation rather than true sensory absence, so an unlisted cell is not treated as proof that the descriptor cannot be perceived.

Label reliability is assessed using reciprocal listing and cross-catalog agreement. These checks measure the consistency of the available records, not their perceptual truth. Reliability-screened training and evaluation use a cell-level eligibility mask: reliable recorded positives and eligible unlisted negatives are included, while ambiguous cells are excluded. This procedure can reduce the influence of uncertain annotations, but it cannot remove every form of catalog bias or labeling error.

The models are evaluated on fixed component-disjoint partitions produced by the clustering and graph-carving procedure described in Chapter 4. Pairs that cross partition boundaries are excluded so that a retained training pair and a retained test pair do not share the same molecule. This design reduces component leakage, but it does not guarantee that the partitions have no structural similarity or that the test molecules represent every possible unseen chemical family [13].

The comparison is limited to the declared model variants, metrics, reliability conditions, and controlled label-corruption experiments. The results therefore show how these methods behave on the selected catalog-derived task and should not be generalized directly to larger mixtures, other descriptor vocabularies, or human sensory judgments. Order-averaged concatenation is an adapted baseline based on the pairwise design of Sisson et al.; it is not a full reproduction of that study’s model. Final dataset sizes, target-vocabulary size, and label density will be reported only after the revised data pipeline has been run.

1.5 Definition of Terms

Molecular Graph

A computational representation of a molecule in which atoms are nodes, bonds are edges, and both carry features used by the graph encoder.

Graph Embedding

A fixed-length numerical vector produced by the graph encoder to summarize the learned information from one molecular graph.

Multi-Label Classification

A prediction task in which one blend can be assigned several descriptors at the same time. Each target descriptor is predicted separately as present or absent.

Mean Pooling

A combination method that takes the element-wise average of the two molecule embeddings. It is naturally independent of input order, but different pairs can produce the same average.

Order-Averaged Concatenation

A combination method that places the two molecule embeddings side by side, evaluates both input orders, and averages the two model outputs so that the final prediction does not depend on which molecule is listed first.

Set Transformer Aggregation

An attention-based method for combining an unordered collection of embeddings. In this study, it learns statistical relationships between the two molecule embeddings before pooling them into one blend representation.

Permutation or Order Invariance

The requirement that exchanging the positions of the two input molecules must not change the model’s final prediction.

Component-Disjoint Split

A data partition in which the same molecule does not occur in both a retained training pair and a retained test pair. It is designed to reduce component leakage, but it does not guarantee that the

partitions contain no structurally similar molecules.

Reliability Mask

A cell-level indicator that determines which blend-descriptor targets are included in reliability-screened training or evaluation. Reliable recorded positives and eligible unlisted negatives are included, while ambiguous cells are excluded.

Evaluation Metrics

Macro F1 gives each retained descriptor equal weight when averaging its F1 score. Micro F1 combines all evaluated decisions before computing one F1 score. Precision-Recall Area Under the Curve (PR-AUC) measures how well the model ranks positive targets across decision thresholds. Hamming loss reports the fraction of evaluated present-or-absent decisions that are incorrect.

Chapter 2

Methodology

This chapter describes the preparation of the catalog-derived datasets, the construction of reliability masks, the formation of component-disjoint data partitions, and the matched training and evaluation of three aggregation methods. It also defines the robustness conditions, metrics, and paired statistical comparisons used to answer the research questions. Because this is a proposal, the procedures are written in the future tense. Settings that have not yet been approved, and numbers that can only be known after the final pipeline is run, are shown as symbols or marked as pending; none of them are results.

2.1 Research Design

This study will use a quantitative experimental design on a multi-label molecular graph classification task. It compares three ways of combining the learned embeddings of the two molecules in a blend: mean pooling, order-averaged concatenation, and Set Transformer aggregation. Fragrance-blend prediction is the application. The computer-science investigation concerns how an unordered pair of graph embeddings should be aggregated, how uncertain labels should be handled, and how models should be evaluated when the two molecules of a pair may also appear in other pairs.

The research questions are the following. **[CONFIRM: copy these word for word from Chapter 1 once both authors approve them.]**

RQ1. Under matched encoding, training, and evaluation conditions, how do mean pooling, order-

averaged concatenation, and Set Transformer aggregation compare in predicting the most reliable catalog-recorded descriptors of two-molecule blends?

RQ2. How does the relative performance of the three aggregation methods vary across catalog-label pattern groups, including additive labels, catalog-label loss, and catalog-label gain?

RQ3. How does the relative performance of the three aggregation methods change when training-label reliability is varied, including under controlled label corruption?

Table 4.1 states what is varied and what is held fixed. The aggregation method is the primary varied factor. Label-reliability and label-corruption conditions are robustness factors, and the catalog-label pattern of a blend is used only for a secondary subgroup analysis. Everything else is a matched control, so that a difference between aggregation methods is not caused by a difference in data, training budget, or luck of the seed.

Table 2.1: Varied factors and matched controls.

Category	Treatment
Aggregation method	Primary varied factor (three levels)
Label-reliability and corruption conditions	Robustness factors
Catalog-label pattern	Secondary subgroup analysis
Encoder design and initialization policy	Matched control
Train/validation/test partitions	Fixed across variants
Loss, optimizer, stopping rule, training budget	Matched controls
Random seeds and corrupted datasets	Paired across variants

The dependent variables are macro F1 (the primary metric, declared before any experiment is run), micro F1, PR-AUC, and Hamming loss, defined in Section 4.7.

2.2 Data Acquisition and Preparation

This section describes the data sources, how they are joined, and how the model inputs (molecular structures) and the model targets (odor descriptor labels) are prepared. The two are prepared separately: structures become graphs, and descriptors become a fixed list of labels.

2.2.1 Data Sources and Provenance

The study will use three sources. The single-molecule sources are GoodScents and Leffingwell, both accessed through Pyrfume. They are independently maintained reference catalogs for the perfumery and flavor industries. Each record lists a molecule, its structure, and the scent descriptors assigned to it (for example, fruity or sweet). The Leffingwell descriptors are described as assigned by olfactory experts, usually practicing perfumers [27]. The blend source is the two-molecule dataset of Sisson et al. [44], obtained from their public repository.

Each Pyrfume source has three files: `behavior.csv`, which holds stimulus–descriptor pairs; `stimuli.csv`, which holds the stimulus identifier, the GoodScents or Leffingwell identifier, the PubChem CID, concentration, and solvent; and `molecules.csv`, which holds the CID, molecular weight, isomeric SMILES, IUPAC name, and name. Descriptors in `behavior.csv` are indexed only by stimulus, so they will first be joined to `stimuli.csv` on the stimulus field to obtain a CID, and then to `molecules.csv` on the CID. Concentration and solvent columns will be dropped, because concentration is outside the scope of this study.

Structure filtering. Before molecules are matched across sources, structures will be sanitized with RDKit: salts are removed, molecules containing elements other than C, H, O, N, S, F, Cl, Br, and I are excluded, and molecules outside a molecular-weight range are excluded. **[PENDING: confirm the element list and the molecular-weight range (20–600 g/mol in the earlier pipeline) and the resulting molecule count, which is a result of the final run and is not stated here.]** This step uses structure only; no label is consulted.

Join key. The two single-molecule catalogs will be merged into one master dictionary using the isomeric SMILES string as the key. Every SMILES string in both sources, and in the blend dataset, will be canonicalized with RDKit so that the same molecule written in two different ways is matched correctly. SMILES was chosen over the PubChem CID because some CIDs in the source files are negative or non-standard and would need further standardization, whereas SMILES must be canonicalized anyway when molecules are converted to graphs. Blend components in the Sisson et al. dataset, which are identified by CID, will be linked to the master dictionary by canonicalizing their SMILES in the same way.

Blend records. The blend dataset of Sisson et al. was built by crawling the “Potential Blenders” section of each GoodScents molecule page, which lists the molecules that molecule could be paired with under scent headings. The labels of a pair are the headings under which the partner was listed. **[CONFIRM: re-read the Sisson et al. Methods and supplement and state exactly how a heading becomes a blend label before this sentence is final.]** Because each molecule’s page was crawled separately, a pair can be listed from both molecules’ pages. The pair-building code of Sisson et al. counts how many pages listed a pair under each heading (one or two) but does not keep the count. This study will keep that count, because it provides the reciprocal evidence used in Section 4.3.

2.2.2 Ground Truth and Its Limits

Catalog-recorded descriptors serve as proxy targets for supervised learning. Reliability procedures estimate the consistency of those records; they do not establish perceptual correctness. The catalogs are curated reference records, not live sensory panel measurements, and their descriptors come from predefined word lists without a reported procedure for combining individual judgments. They therefore contain some noise and subjectivity [6]. This study treats them as a practical proxy for perceived scent, estimates how consistent they are, and examines how model comparisons change as label reliability changes.

For the computational scope of this study, each blend is treated as a simple physical mixture of two aroma chemicals. No significant intermolecular reaction is expected for most pairs under the assumed blending conditions (room temperature, no catalyst), so the data are treated as standard fragrance blending rather than reaction chemistry. A few highly reactive pairs may deviate from this expectation. **[CONFIRM: wording comes from chemistry faculty; permission to name the source is pending.]**

2.2.3 Target Vocabulary

Two vocabularies must be kept apart. The *standardized vocabulary* is the list of descriptor names after spelling variants have been merged. The *final target vocabulary*, of size L , is the subset of the standardized vocabulary on which models are actually trained and scored.

Standardization. The raw GoodScents and Leffingwell labels contain spelling variants of the same descriptor (for example, anisic and anise, wine and winey, or caramel and caramelic). Reviewing every label by hand would be impractical, so the study will adopt the open-source canonicalization dictionary of Sisson et al. [44], which merges raw blend descriptors into 104 standardized descriptors. **[CONFIRM: the source text gives both “approximately 400 raw descriptors” and “109 raw notes” for the input to this step; check which is correct and use one.]** Any descriptor in the merged data that the dictionary does not cover will be reviewed manually and either mapped to an existing descriptor or declared new, followed by whitespace and punctuation normalization. The standardized single-molecule vocabulary may exceed 104 descriptors if new descriptors survive this review; Appendix B will list it.

Final target vocabulary. The final target vocabulary will be built from *training data only* (Section 4.5.4). Validation and test labels will not be consulted when deciding which descriptors are kept. The size L is therefore not known yet and is written as a symbol throughout. The figure 104 is the size of the standardized vocabulary before partitioning, and the figure 74 is the number Sisson et al. reported after their own partitioning; neither is assumed to be this study’s L .

Inputs and targets. Input preparation (canonical SMILES to molecular graph with atom and bond features, Section 4.4.1) uses only molecular structure. Target preparation (descriptor standardization, mask construction, vocabulary selection) uses only descriptor labels. Neither step uses the test partition to make a decision.

2.2.4 Label Families (Reporting Only)

Expert-curated odor taxonomies, such as that of Sajan et al. [41], group raw odor words into broader families (for example, rose, jasmine, and violet under floral). The model will still predict each descriptor individually, because catalog-label loss and gain refer to specific words rather than to families. The Sajan et al. grouping is used for organized reporting only: Appendix B will map each descriptor to a family and give the number of descriptors per family. No per-family scores will be computed.

2.3 Label Reliability Mask

Not every recorded label is equally trustworthy, and not every missing label is a true absence. Rather than assigning one reliability grade to a whole blend, the study will decide, for each *blend-descriptor cell*, whether that cell is eligible for a stricter, “Gold-only” treatment. A cell is one entry of the blend-by-descriptor label matrix: one blend and one descriptor.

2.3.1 Consistency Evidence

Two kinds of evidence are used. Both measure consistency, not correctness.

- **Reciprocal-positive evidence (cell level).** A recorded positive is *reciprocal* if the pair was listed under that descriptor from the pages of both molecules (a count of two). Using the pair-building code of Sisson et al. [44], a preliminary audit found that 61,838 of 235,040 listings (26.3%) were reciprocal. **[CONFIRM: this is a preliminary audit figure; recompute with the final pipeline before reporting.]** A one-sided listing is not necessarily wrong, since catalogs do not list every possible blend, but it is not independently confirmed.
- **Descriptor-level consistency (descriptor level).** Agreement between GoodScents and Leffingwell on the molecules they share will be measured separately for each descriptor using Cohen’s kappa. In a preliminary audit on 2,225 shared molecules, 64 of 104 descriptors reached a kappa of at least 0.21 (“fair” on the scale of Landis and Koch). **[PENDING: the statistic and the cutoff are not final. The team must approve the statistic, the cutoff, and the molecule set used to compute it.]** A descriptor that meets the cutoff is called *consistent*. Reaching the cutoff does not prove that the labels are correct.

2.3.2 Eligibility Mask

Table 4.2 defines the eligibility mask $M_{ij} \in \{0, 1\}$ for blend i and descriptor j in the Gold-only treatment.

Because eligible unlisted descriptors enter as negatives, a Gold-only test set is not positive-only: a model that predicts a descriptor that is not recorded for a blend is still penalized as a false positive.

Closed-world assumption. Treating an unlisted descriptor as a negative is an operational assumption. It is needed because supervised multi-label learning requires a definition of “absent.”

Table 2.2: Eligibility of a blend–descriptor cell in the Gold-only treatment.

Cell evidence	Descriptor is consistent	Gold-only treatment
Reciprocal recorded positive	Yes	Include as positive ($M_{ij} = 1$)
One-sided or otherwise ambiguous recorded positive	Either	Mask ($M_{ij} = 0$)
Unlisted descriptor	Yes	Include as negative ($M_{ij} = 1$), under the closed-world assumption
Any cell	No	Mask ($M_{ij} = 0$)

Catalog omission may not mean true sensory absence, since a catalog may simply not list a blend or a descriptor. The reliability mask reduces this uncertainty but does not remove it, and results will be interpreted with this limitation in mind.

2.3.3 Experimental Label Conditions

Two label conditions are used.

- **Full-label condition.** Every cell is used ($M_{ij} = 1$ for all i, j), with listed descriptors as positives and unlisted descriptors as negatives.
- **Gold-only condition.** Only cells with $M_{ij} = 1$ under Table 4.2 are used.

Masked loss. In the Gold-only condition a blend is kept as long as it has at least one eligible cell; it is not removed merely because some of its cells are ambiguous. Loss is computed only over eligible cells:

$$\mathcal{L}_{\text{masked}} = \frac{\sum_i \sum_j M_{ij} \ell_{ij}}{\sum_i \sum_j M_{ij}} \quad (2.1)$$

where ℓ_{ij} is the per-cell loss (Section 4.4.3). In the full-label condition $M_{ij} = 1$ everywhere and the expression reduces to the ordinary mean loss. The same mask is applied when Gold-only metrics are computed (Section 4.7). The training, validation, and test partitions each carry their own mask, built from the same rule.

Insufficient support. After masking, a descriptor may have no eligible positives, or no eligible negatives, in a partition. F1 and PR-AUC are undefined for such a descriptor. Any descriptor with fewer than s eligible positives or fewer than s eligible negatives in the evaluated partition will

be excluded from macro-averaged scores, and the excluded descriptors will be listed in the report. **[PENDING: the minimum support s must be fixed before final evaluation.]** A training run that has no eligible cells at all is reported as a failed run and is not silently dropped.

2.4 Model Architecture

All three models share one molecular graph encoder and one output head. They differ in the step that combines the two molecule embeddings. The data flow is: each molecule is turned into a graph and encoded into a vector; the two vectors are combined by the aggregation method; and the combined vector is converted into one probability per descriptor.

2.4.1 Shared Graph Encoder

Each molecule is represented as a graph in which atoms are nodes and bonds are edges. A Graph Attention Network (GAT) encoder turns each graph into a fixed-length vector $\mathbf{z} \in \mathbb{R}^d$, so a blend yields two embeddings $\mathbf{z}_A$ and $\mathbf{z}_B$. The same encoder, with the same weights, is applied to both molecules.

[CONFIRM: node feature size, edge feature size, number of GAT layers, attention heads, readout, and embedding size d must be copied from Chapter 3 after the revised architecture is approved. The values used in the earlier notebook (46 node features, 10 edge features, 3 layers, $d = 256$) are not inherited automatically.] The attention computation is defined in Chapter 3 **[TODO: replace with \ref to the Chapter 3 section label]**. In the rest of this chapter d stands for the embedding size and, where a concrete number is needed for illustration, $d = 256$ is used.

Pretraining. The encoder may first be pretrained on single-molecule data, using a temporary head with one sigmoid output per standardized single-molecule descriptor and binary cross-entropy loss. Only molecules assigned to the training partition (Section 4.5) will be used, so molecules reserved for validation and testing are never seen during pretraining. After pretraining the temporary head is discarded and only the encoder weights are kept.

Encoder policy. **[PENDING: the team must choose one policy for the final comparison.]** The earlier notebook’s freeze/unfreeze schedule is implementation experience only and is not

the revised protocol. The two options are:

- a *frozen shared encoder*, in which encoder weights are fixed after pretraining and only the aggregation module and output head are trained; or
- the *same pretrained initialization followed by matched end-to-end fine-tuning* for all variants.

If the second option is chosen, the text will state: “The variants begin from the same encoder initialization and use matched schedules and optimization settings. Their encoder weights may diverge during end-to-end training.” In that case this study will not claim that the final models differ only in how they combine embeddings, because their encoders can end up different. Either way, the initialization policy, schedule, and training budget will be identical across the three aggregation methods.

2.4.2 Aggregation Variants

Table 4.3 summarizes the three variants. For two molecules, the embeddings form an unordered pair $\{\mathbf{z}_A, \mathbf{z}_B\}$, and a blend prediction should not change if the two molecules are swapped (permutation invariance).

Table 2.3: The three aggregation variants.

Variant	General operation	Order handling
Mean pooling	Average the two molecule embeddings	Invariant by construction
Order-averaged concatenation	Run both ordered concatenations through the head and average the outputs	Explicit symmetrization
Set Transformer aggregation	Apply set attention, then invariant pooling	Invariant set processing

Mean pooling. The blend embedding is the unweighted average of the two molecule embeddings,

$$\mathbf{z}_{\text{blend}} = \frac{1}{2}(\mathbf{z}_A + \mathbf{z}_B) \in \mathbb{R}^d. \quad (2.2)$$

This is simple and invariant to order. Sum pooling is not included as a separate variant: for exactly two molecules the sum is twice the mean, so it adds no information once the output head is trainable.

Order-averaged concatenation. The two embeddings are placed side by side, $[\mathbf{z}_A | \mathbf{z}_B] \in \mathbb{R}^{2d}$ (512 dimensions when $d = 256$). Raw concatenation depends on the order of its inputs, so the model is evaluated on both (A, B) and (B, A) and the two outputs are averaged. This follows the pairwise design of Sisson et al. [44], but it is an *adapted baseline*, not a reproduction of their whole model: it uses this study’s encoder, loss, and partitions. **[PENDING: state whether the averaging is applied to logits or to probabilities, and use the same choice in every chapter and slide. Averaging logits and applying the sigmoid once is the simpler option, but the team must decide.]**

Set Transformer aggregation. The pair $\{\mathbf{z}_A, \mathbf{z}_B\}$ is treated as a set and passed through a Set Attention Block (SAB), in which each embedding attends to both embeddings in the set (itself included):

$$\text{SAB}(X) = \text{MultiHead}(X, X, X), \quad (2.3)$$

where queries, keys, and values all come from the same set X . This lets the representation of each molecule be adjusted using the other before the two are combined. The attention mechanism follows the definition in Chapter 3 **[TODO: \ref]**. The adjusted pair is then reduced to a single blend embedding $\mathbf{z}_{\text{blend}} \in \mathbb{R}^d$ by Pooling by Multihead Attention (PMA), which uses learned attention weights instead of a plain average. **[PENDING: number of SAB blocks, number of attention heads, hidden size, and whether PMA uses one seed vector; values from the earlier notebook (2 blocks, 4 heads) are not inherited automatically.]** Both SAB and PMA are permutation-invariant in the sense needed here: swapping $\mathbf{z}_A$ and $\mathbf{z}_B$ does not change $\mathbf{z}_{\text{blend}}$. This module is a learned way of mixing two embeddings. This study does not claim that it models physical or receptor-level interaction between the molecules.

Complexity. Once the models are implemented, the study will report the number of trainable parameters of each full model and a measure of computation cost (for example, training time per epoch and inference time per batch on the same hardware), so that any accuracy difference can be read together with the cost of obtaining it. **[FILL IN: parameter counts and timings.]**

2.4.3 Output Head and Loss

The blend embedding is passed through a Multi-Layer Perceptron that maps $d \rightarrow h \rightarrow L$, where h is a hidden width and L is the final target vocabulary size (Section 4.2.3). **[PENDING: h and L ; the earlier notebook used $h = 128$.]** For order-averaged concatenation the first layer takes $2d$ inputs. The output layer applies an independent sigmoid to each of the L nodes, because odor descriptors are multi-label: each descriptor is its own present/absent decision, unlike softmax, which forces the outputs to sum to one. The framework for multi-label learning and the Asymmetric Focal Loss are defined in Chapter 3 **[TODO: \ref]**. Following Hatami and Moradi [19], the loss uses a focusing parameter of $\gamma_p = 0$ for positives and $\gamma_n = 2$ for negatives **[PENDING: confirm these values for the revised study]**. The per-cell loss ℓ_{ij} of this function enters the masked loss of Section 4.3. All three variants use the same loss, optimizer, learning-rate schedule, early-stopping rule, and training budget.

2.5 Component-Disjoint Partitioning and Graph Carving

This section describes how the data are divided into training, validation, and test sets. The aim is that no molecule that appears in a training blend also appears in a test blend. This section describes the procedure; it does not claim that the test set contains only chemically unfamiliar molecules.

Why random pair splitting leaks components. A blend dataset is a collection of pairs, and the same molecule appears in many pairs. If pairs are split at random, a molecule seen in a training pair is very likely to appear again in a test pair, so test performance partly measures memorization of molecules rather than prediction for new ones. Group-aware splitting is the standard remedy for this kind of overlap [24] (see Section 2.13).

2.5.1 Scaffold Split Decay

Scaffold splitting, which groups molecules by shared core framework, is often used for this purpose. In sparse chemical spaces, however, most scaffold groups contain very few molecules, and scaffold splitting then behaves much like random splitting (Section 2.13). To check this on the present data,

the molecule-to-scaffold ratio will be computed:

$$R = \frac{N_{\text{molecules}}}{N_{\text{scaffolds}}} \quad (2.4)$$

where $N_{\text{molecules}}$ is the number of unique canonical molecules and $N_{\text{scaffolds}}$ is the number of distinct Bemis–Murcko frameworks produced by RDKit. A value of R near 1 means almost every scaffold holds a single molecule. R will be computed on the merged master dictionary and compared with the collapse threshold $R < 2.0$ reported by Fooladi et al. [13].

2.5.2 Cluster-Based Partitions

Molecules will be grouped by structural similarity and whole groups will be assigned to one partition. The planned pipeline is: compute a binary structural fingerprint for every unique molecule; reduce it to a low-dimensional map with UMAP; and cluster the map with K-Means into $k = 7$ clusters. The number of clusters is fixed at seven, following Guo et al. [16], who found seven UMAP clusters to outperform alternative clustering methods on molecular data. A candidate fingerprint is ECFP4 (2048 bits, radius 2).

[PENDING: fingerprint type and size, UMAP settings, random seeds, and the cluster-to-partition rule are not final. They will be fixed and written down before any final evaluation, and the values of the earlier notebook will be reused only if the team deliberately re-approves them. The value $k = 7$ is the only clustering setting treated as fixed.]

The rule that assigns clusters to training, validation, and test partitions will be *declared in advance and will not depend on any label*. For example, it may depend only on cluster sizes (such as assigning clusters to partitions so that the molecule counts approximate a target ratio). **[PENDING: state the exact rule.]** The selection will not look at which descriptors occur in which cluster, and it will not look at test labels. Once the partitions are fixed they are not changed; the number of molecules, pairs, and descriptor support in each partition are reported afterwards as outcomes, not used as selection criteria. If any inspection of test labels turns out to be unavoidable, it will be disclosed, and the test set will not be described as untouched.

2.5.3 The Graph-Carving Protocol

The blend dataset is treated as a graph in which each node is a unique canonical molecule and each edge is a documented blend between two molecules. Every molecule is assigned to a partition by its cluster. Carving then examines each edge: an edge (blend) is *retained only if both molecules belong to the same assigned partition*, and discarded otherwise. After carving, no molecule is shared between any two partitions, so no molecule component can leak from training into validation or test. Sisson et al. [44] applied a similar carving to a random 50:50 molecule partition, which discarded about 83,000 pairs and left about 44,000 training and 40,000 testing pairs. This study uses clusters instead of a random molecule split, so its pair counts will differ and are reported only after the procedure is run. **[FILL IN: pairs retained and discarded per partition.]**

The validation set is used for early stopping, hyperparameter decisions, and selection of decision thresholds. The test set is kept sealed and used only in the final evaluation.

Component-disjoint partitioning reduces component and structural overlap between partitions. It does not guarantee that test molecules belong to unfamiliar chemical families, because clusters that are near each other on the map can still contain similar molecules.

2.5.4 Training-Only Vocabulary Filter

Carving changes how often each descriptor occurs, so the final target vocabulary is chosen after partitioning, from training data only. A standardized descriptor d is kept if it has enough positive examples to learn from:

$$d \in \mathcal{D}_{\text{target}} \iff c_{\text{train}}(d) \geq x \quad (2.5)$$

where $c_{\text{train}}(d)$ is the number of training blends in which d is recorded as positive, and x is a minimum-support threshold. **[PENDING: the value of x ; the earlier notebook used $x = 5$ on both training and test sides, and the test-side condition is removed here.]** The size of $\mathcal{D}_{\text{target}}$ is L . In the Gold-only condition, $c_{\text{train}}(d)$ counts eligible positives. After the vocabulary is locked, the number of validation and test positives and negatives for each descriptor is *reported* but is not used to add or remove descriptors. Descriptors that end up with too little test support are handled by the insufficient-support rule of Section 4.3 and are listed in the report.

2.6 Experiments

All experiments hold the partitions, the encoder initialization policy, the loss, the optimizer, the stopping rule, the training budget, and the metric suite constant across the three aggregation methods. They will show whether attention-based aggregation performs better than simpler alternatives; a finding that a simpler baseline performs as well or better is a valid and informative result and is reported as such. Each configuration is run with the same set of random seeds for every aggregation method. **[PENDING: the number of seeds.]**

2.6.1 Experiments A and B: Main Comparison (RQ1)

Experiment A trains each model in the full-label condition, using all cells. Experiment B trains each model in the Gold-only condition, using masked loss over eligible cells (Section 4.3). Both are evaluated on the held-out test partition. Experiment A is also scored on all cells, and Experiment B is scored on eligible cells with the same mask. Reporting both makes visible how much of any difference depends on which cells are counted.

2.6.2 Experiment C: Controlled Label Corruption (RQ3)

To examine how the comparison changes as training labels become less reliable, labels will be corrupted on purpose, in the training partition only. Validation and test labels are never changed.

For a noise level $p \in \{0, 0.10, 0.20, 0.30\}$, a fraction p of the eligible positive cells in the training labels is flipped to absent, and an equal number of eligible negative cells is flipped to present, which keeps the density of positives unchanged. Only eligible cells can be flipped, so the same mask logic as Section 4.3 applies. For each combination of noise level and random seed, one corruption mask is generated and *reused across all three aggregation methods*; a new mask is generated for every seed and noise level. The run at $p = 0$ is the uncorrupted Gold-only run of Experiment B and is reused. Models are retrained at each level and scored on the unchanged validation and test labels. The change in macro F1 from $p = 0$ to $p = 0.30$ is then compared across methods using the paired bootstrap of Section 4.8.

Random flipping tests robustness to label errors in general. It does not reproduce every real process by which catalog errors arise.

2.6.3 Blend-Type Analysis (RQ2)

To see whether the relative performance of the methods differs by catalog-label pattern, each test blend is compared with the recorded descriptors of its two parent molecules. The terms below describe patterns in the catalog records; they are not claims about what a person would perceive.

- **Catalog-label loss:** a descriptor recorded for either parent is not recorded for the blend.
- **Catalog-label gain:** the blend has a recorded descriptor that is not recorded for either parent.
- **Additive:** the blend shows neither loss nor gain.

Descriptive statistics of the label matrix (label density, common co-occurrences, and the rate at which a blend’s labels are additive with respect to its parents) are reported for the dataset as a whole. They are used to describe the data and to support this subgroup analysis; they are not used to filter data. The sizes of these three groups are reported and the model comparison is repeated within each group. Because these groups overlap in how they are defined and the analysis is split into subgroups, it is treated as a secondary subgroup and error analysis rather than a confirmatory test. If any group contains fewer than 5% or more than 95% of the test blends, blends are instead divided by the overlap between the blend descriptors and the union of the parents’ descriptors.

2.7 Evaluation Metrics

This section defines the metrics and explains why each is needed when most label cells are negative.

2.7.1 Multi-Label Metrics

Let TP be a descriptor correctly predicted present, FP a descriptor incorrectly predicted present, FN a present descriptor that was missed, and TN a descriptor correctly predicted absent. Most cells in the label matrix are negative, so a model can look accurate by predicting “absent” every time. Precision, recall, and F_1 use only TP, FP, and FN [37], so they ignore the many easy true negatives, and a model that predicts nothing receives $F_1 = 0$.

- **Macro F1 (primary metric).** F_1 is computed for each descriptor separately and then averaged with equal weight, so rare descriptors count as much as common ones [37]. This choice

is declared before experimentation and must be the same in Chapters 1–4. [**CONFIRM: Chapter 1.**]

- **Micro F1.** TP, FP, and FN are pooled across all descriptors before F_1 is computed; it summarizes overall performance and favors common descriptors.
- **PR-AUC.** The area under the precision–recall curve, which measures how well the model ranks true descriptors above false ones across all thresholds. It is reported both macro-averaged over descriptors (primary version) and micro-averaged. [**PENDING: confirm that both versions are reported.**]
- **Hamming loss.** The fraction of evaluated cells that are predicted incorrectly (below).

Thresholds. Each descriptor’s probability is turned into a present/absent prediction with a decision threshold. Thresholds are chosen on validation data only and then applied unchanged to the test data. Threshold selection matters a good deal for imbalanced multi-label problems [54]. [**PENDING: whether one global threshold or a per-descriptor threshold is used.**]

2.7.2 Masked Metrics

All metrics are computed only over cells with $M_{ij} = 1$, using the mask of the condition being evaluated (all ones in the full-label condition). For macro metrics, descriptors with insufficient eligible support are excluded and listed (Section 4.3). With a mask, Hamming loss is

$$\text{HL}_M = \frac{\sum_{i=1}^N \sum_{j=1}^L M_{ij} \mathbf{1}(y_{ij} \neq \hat{y}_{ij})}{\sum_{i=1}^N \sum_{j=1}^L M_{ij}} \quad (2.6)$$

where N is the number of evaluated blends, L is the number of target descriptors, y_{ij} is the label, $\hat{y}_{ij} \in \{0, 1\}$ is the thresholded prediction, and $\mathbf{1}(\cdot)$ equals 1 if its argument is true and 0 otherwise. Smaller values are better, with $\text{HL}_M = 0$ for a perfect match. In the full-label condition $M_{ij} = 1$ everywhere and this is the ordinary Hamming loss.

All-negative reference. Hamming loss also rewards the many easy negatives, so it cannot be read alone. The *evaluated-cell density* is

$$D_M = \frac{\sum_i \sum_j M_{ij} y_{ij}}{\sum_i \sum_j M_{ij}} \quad (2.7)$$

the fraction of evaluated cells that are positive. A model that predicts every cell absent has $HL_M = D_M$, which serves as the all-negative reference. It is computed separately for each partition (D_{train} , D_{val} , D_{test}) and for each mask, and a value from one setting is never reused in another. **[FILL IN: densities and the all-negative reference, after the pipeline is run.]** Every model’s Hamming loss is reported next to this reference.

2.8 Statistical Comparison

Differences between models could reflect chance in which blends happen to be in the test set, or chance in training. These two sources are handled separately.

Variability across seeds. Optimization variability comes from random initialization and data order. Each model is trained with the same set of random seeds, and results are summarized across seeds (mean and spread) separately from the bootstrap below.

Bootstrap over blends. Descriptor outcomes within one blend are correlated, so individual cells are not independent. The test blends are therefore resampled as whole rows: a sampled blend keeps all of its descriptor cells together, with its mask. Naive paired t -tests are not appropriate for comparing machine-learning models, because they are sensitive to outliers and not valid when resampled test sets are used [37], and bootstrap resampling shows both whether a difference exists and how large it is [5].

From the N test blends, N rows are drawn with replacement, B times **[PENDING: the number of replicates B will be fixed before final evaluation]**. All models being compared are scored on the *same* resampled indices, giving a paired comparison. For replicate b the paired difference is

$$\Delta(b) = \text{Metric}_{\text{model 1}}(b) - \text{Metric}_{\text{model 2}}(b). \quad (2.8)$$

The direction of a difference is stated with each result: higher is better for macro and micro F1 and for PR-AUC, and lower is better for Hamming loss, so a positive Δ favors model 1 for the first three and model 2 for Hamming loss. The comparison is repeated for each metric and for each pair of aggregation methods (three pairs). **[PENDING: whether a correction for multiple comparisons is applied, or whether the primary comparison is limited to macro F1.]**

Intervals and effect size. The significance level is $\alpha = 0.05$. A 95% percentile confidence interval is taken from the 2.5th and 97.5th percentiles of the B differences, and an interval that excludes zero is read as a difference at $\alpha = 0.05$. Because statistical significance does not show whether a difference matters in practice, the effect size is reported as the mean difference with its 95% interval, in the units of each metric. For Experiment C, the bootstrap is applied to the change in macro F1 between noise levels.

2.9 Reporting Conventions

Experiments that have not been run are described in the future tense. Any pipeline demonstration run before the final protocol is fixed will be labelled preliminary and will not be reported as a result. Output from the earlier notebook is not reported as a result of the revised study. The study does not assume that the Set Transformer will outperform the baselines; if mean pooling or order-averaged concatenation matches or exceeds it, that outcome will be reported and discussed on equal footing. Quantitative Structure-Odor Relationship (QSOR) modeling refers to the prediction of the odors and olfactory properties of molecules through the characterization of their structure [15, 56, 61]. It has long been regarded as one of the key challenges in olfactory research in machine learning [7, 32, 58]. QSOR is especially important in fields such as the fragrance and flavor industries, as it could help streamline the formulation and development of new compounds computationally, as opposed to traditional experimental wet lab methods, which can be both time-consuming and expensive [20, 40].

Early approaches in QSOR involved the manual creation of numerical features that describe and quantify the properties of a molecule [3, 15, 62]. These included molecular fingerprints, which encode the presence or absence of substructures in a binary vector, and physicochemical descriptors computed with dedicated software [3, 11, 42, 55].

Many studies pair such descriptors or fingerprints with classical learners such as random forest, XGBoost, and LightGBM to predict odor [40, 48, 54]. Two comparisons are representative. Suh et al. found that Morgan-fingerprint models outperformed descriptor-based ones on a curated set of 8,681 fragrance compounds [48], and Achebouche et al. found that graph convolutional representations outperformed general molecular descriptors and fingerprints on a set of 5,955 molecules [1].

The common denominator across these approaches is that they convert the molecule, specifically

its properties, into fixed, handcrafted numerical representations, which are then fed into classical ML models. Recent work such as Suh et al., Achebouché et al., and Wen et al. continues to include descriptor-based classical models, but mainly as baselines to quantify how much fingerprint- and graph-based learning improves upon manual feature engineering [1, 48, 54]. While effective, these methods across both eras require the manual selection of descriptors in advance [4, 27, 62]. This means the model is not actually able to determine on its own which structural features are relevant for the task it is trying to solve. This necessitated the advancement to graph-based deep learning methods, which improve the odor prediction pipeline by teaching models to learn task-relevant features directly from structure [2, 4, 18, 61], which are discussed next.

2.9.1 The Principal Odor Map

Addressing the aforementioned limitation is Lee et al.’s Principal Odor Map (POM), which maps a molecule’s chemical structure to its smell using Graph Neural Network (GNN) techniques [27]. Rather than evaluating a molecule’s scent using a static representation built from predefined handcrafted descriptors, POM learns directly from the molecule’s structure during training. It does this by treating each molecule as a graph, where nodes and edges represent atoms and chemical bonds, respectively; each node and edge is itself a vector, with nodes storing properties such as atomic number, and edges storing bond properties.

Through message passing, the GNN dynamically assigns weights to each of the molecule’s structural features across multiple passes, producing a numerical representation of the molecule. The final layer predicts the molecule’s specific odor qualities, while the penultimate layer forms the actual Principal Odor Map, which the paper itself describes as a high-dimensional, data-driven representation that is able to preserve human perceptual distances and hierarchies, positioning molecules with similar smells closer together in the generated odor space [27].

The setup was as follows: Lee et al. used approximately 5,000 molecules derived from merging data from the GoodScents and Leffingwell fragrance databases to produce a multi-label output of 138 distinct semantic odor descriptors. It was then validated against a human benchmark, wherein approximately 15 trained panelists rated molecules using a narrowed-down set of 55 odor descriptors. The results show it was able to match the trained panel’s verdicts on every label except for musky scents, and was even able to surpass the median panelist on 30 out of the 55 validation labels (55%) [27].

This demonstrates that graph representations can encode predictive scent information from molecular structure alone. However, despite this significant advancement, these results were derived exclusively from single, unmixed molecules.

2.9.2 Limitations of Single-Molecule Models

POM shows what a graph encoder can learn for one molecule, but it operates on a single molecule at a time, while odorants rarely appear in isolation [9, 27, 51, 59]. Real-world scents are rarely composed of single molecules; rather, they are blends that can contain individual components numbering up to the hundreds [17, 59]. The limitation is computational: a single-graph encoder produces one vector per molecule, and it does not define how the vectors of two molecules should be combined into one prediction. That combining step is the subject of this study.

Lee et al. admit this limitation themselves, noting that while POM uses a coordinate system to categorize scents, it alone cannot determine the behavior and outcomes of different molecules when mixed [27].

2.10 Psychophysics of Odor Mixtures

Because single-molecule models do not specify how two molecules are combined, a method for combining them is needed. This section establishes only that both simple averaging and interaction-aware aggregation are plausible candidates for that method. It reviews how catalog blend labels differ from the labels of their parent molecules, and the human-panel evidence that averaging may be sufficient. It does not claim that either method reproduces perception.

2.10.1 Scent Blending Mechanics

A paper released by Sisson et al. introduces a labeled dataset consisting of molecule pairs to measure the divergence of the emerging scent when two molecules are mixed from their original scent labels, using two metrics: union and intersection. The union score operates under the assumption that the emerging scent will smell like any of the scents either parent molecule has (additive mixing). On the other hand, the intersection score operates under the assumption that the outcome of the two molecules when mixed will only come from the scents the parent molecules share (subtractive mixing) [44].

The Jaccard results show 0.12 for union and 0.24 for intersection [44]. These low scores indicate that neither additive nor subtractive mixing reflects the catalog blend labels. This suggests that simple union or intersection rules do not reproduce the catalog-recorded labels of molecule pairs.

Catalog blend labels differ from the labels of the parent molecules in two ways. In *catalog-label loss*, a scent word recorded for either parent molecule is absent from the blend’s labels. In *catalog-label gain*, the blend’s labels contain a scent word that neither parent has. The results of Sisson et al. show that, on average, 1.2 notes are lost and 0.2 notes are gained per blend. Furthermore, an odds-ratio analysis reveals that certain scents, such as “eggy” or “cabbage”, were more likely to be gained, while certain scents like “peach” or “fresh” were more likely to be lost [44]. These are differences between catalog records, not verified perceptual effects. They appear to follow a pattern rather than being random, which motivates testing a model that can learn relationships between the two molecules, such as a Set Transformer.

2.10.2 Evidence That Averaging May Be Sufficient

Not all evidence points to strongly non-additive mixing. Pellegrino et al. [35] had a trained panel rate 432 mixtures using a 51-term list and found that a linear average of the components’ ratings predicted mixture profiles about as well as the noise in the panel’s own ratings allowed. They argue that apparent emergence often reflects the limits of describing an odor with a few words. Binary catalog labels, as in Sisson et al. [44], instead record catalog-label loss and gain in many blends. Whether an interaction step helps for binary multi-label prediction is therefore an open question, which this study tests rather than assumes.

Human studies also describe two ways in which a mixture can be perceived: elementally, where its components can be picked out, and configurally, where the mixture has a quality of its own [10, 39]. Which one dominates can depend on the proportions of the components, and because this study does not model concentration, each blend is treated as an unweighted pair. Odorant antagonism and masking have been demonstrated for some odorants [14, 22, 23]. This study does not model receptors; the Set Transformer is evaluated only as a statistical aggregator of two learned vectors.

2.11 Machine Learning on Sets and Chemical Mixtures

This section is the core of the chapter. It defines permutation invariance, explains how mean pooling, concatenation, and the Set Transformer each combine two molecule vectors, and reviews prior work on mixture modeling.

2.11.1 Permutation Invariance on Sets

A combination of two or more molecules could be viewed as a mathematical set. An accurate depiction of a fragrance blend of these molecules requires that they be presented in such a way that their positioning in that list is arbitrary, not sequential [44, 51, 61]. For instance, assume the combination of molecule A and molecule B produces scent C. It should be of no relevance whether it is molecule A + B or molecule B + A; it should always produce scent C. For that reason, it is important for any mathematical model used to predict the resultant smell from the combination of different molecules to follow this principle, which is called permutation invariance. In plain terms, a function is permutation invariant if swapping its inputs does not change its output.

Zaheer et al. [60] established the math of permutation invariance and introduced Deep Sets, a deep learning framework that uses a neural network designed to process datasets in a non-sequential manner for unordered data, which serves a different function from architectures that must process data in order (e.g., molecular compounds vs. protein sequences). This is referenced as the first of two properties they introduced in their study describing permutation invariance, positing that for a function acting upon a set, it must be indifferent to the order in which each element appears, mathematically formulated as $f : 2^X \rightarrow Y$ for any permutation, i.e., for any permutation π :

$$f(\{x_1, \dots, x_M\}) = f(\{x_{\pi(1)}, \dots, x_{\pi(M)}\}).$$

Their second result explains how to build such a function: transform each element individually (ϕ), aggregate the results, and then transform the aggregate again (ρ) [60].

However, because each element is transformed before pooling and in isolation, any information about how the elements relate to each other must be inferred indirectly by later layers from the pooled representation, as noted by Lee et al. [28]. For a pair of molecules, this raises the question of whether letting the two vectors affect each other before pooling helps.

To bridge this gap, Lee et al. propose the use of a Set Transformer, where instead of processing the elements of a set individually, they apply self-attention mechanisms they call SAB (self-attention

block) in the processing of the elements, wherein the different elements are able to learn from each other, such that the extracted features to be used for pooling already contain useful information about the elements’ relationship with each other. Moreover, they also developed PMA, or Pooling by Multihead Attention, which replaces the second step in the Deep Sets architecture wherein, instead of fixed sum or mean pooling, PMA also employs self-attention mechanisms in the pooling itself, such that it is able to pick out which of the elements bear more weight for the task at hand. Both are candidates for combining the two molecule vectors in this study: SAB lets each vector be updated using the other before pooling, and PMA learns how to weight them. The Set Transformer is therefore treated as a way to model statistical relationships between two learned vectors, not as a model of masking, emergence, or receptor behavior.

Three ways of combining the two vectors are compared in this study. *Mean pooling* averages the two vectors. It is simple and naturally order independent, but it can lose information specific to the pair, because different pairs with the same average produce the same combined representation [49]; this does not mean that it must fail on any particular task. *Concatenation* places the two embeddings side by side, as in the pairwise model of Sisson et al. [44]. It keeps both embeddings in full, but its output depends on which molecule is listed first. This study therefore symmetrizes it by evaluating both orders and averaging the two outputs, which is called *order-averaged concatenation*. *Set Transformer aggregation* lets the two vectors affect each other’s learned representation before pooling, while remaining independent of order.

The three methods also differ in cost. Self-attention blocks in the Set Transformer scale quadratically with the number of elements in a set [28]; for a set of two molecules this cost is small, but attention blocks still add trainable parameters and computation compared with fixed pooling. A fair comparison therefore reports parameter counts and computation cost alongside accuracy, so that any gain can be weighed against the extra complexity.

2.11.2 Precedents in Mixture Modeling

Zaheer et al. [60] provided the groundwork for permutation invariance in sets, while Lee et al. [28] extended it with self-attention, a general set method that later studies applied to molecular mixtures. Plenty of studies have made use of their work in different realms of chemistry.

For example, Qin et al. [36] built SolvGNN to predict activity coefficients of binary and ternary chemical mixtures using two levels of graph processing: graph convolutions inside each individual

molecule, followed by a molecule-level graph whose nodes are the molecules themselves. They argue that modeling the mixture explicitly at this second level matters in cases where naive pooling falls short [36].

Similarly, Sanchez Medina et al. [42] introduce GH-GNN, which also predicts activity coefficients of chemical mixtures. It uses a single shared molecular encoder whose pooled output is passed to a mixture-level graph. This resembles the structure of the present study, a shared single-molecule encoder followed by a mixture-level step, but applied here to odorant pairs rather than to solvent and solute.

Lastly, Tom et al. [51] proposed POMMix, a more relevant precedent for our topic, as it is in the field of olfactory perception, aiming to predict the similarity of the scent of two mixtures. Theirs employs a 3-layer architecture, with the first making use of the previously discussed POM, where a GNN processes the atomic bond traits of the molecules along with the scent labels to create a multi-dimensional vector that describes the scent of the molecule. The second layer employs a module they call CHEMIX, which uses a self-attention mechanism across the graph from the first layer, doing set-pooling to produce a permutation-invariant mixture embedding that is able to tell which components contribute to the nonlinear behaviors that odorants exhibit. The last layer of their architecture is the cosine predictive head, which evaluates the similarity of mixtures according to their scent by calculating the scaled cosine distance between their respective embeddings. Their ablation study compared several molecular embedding types including handcrafted RDKit descriptors and pretrained language-model embeddings, combined with both a classical model (XGBoost) and their attention-based CHEMIX module, finding that embeddings derived from POM consistently outperformed the alternatives, suggesting that structure-aware, attention-based representations are useful for mixture modeling. POMMix differs from this study in its prediction task: it predicts the similarity between two mixtures, whereas this study predicts the scent labels of a two-molecule blend.

These three studies tackle different prediction problems. Evidence from solvent mixtures in particular does not transfer directly to scent labels. They motivate evaluating interaction-aware aggregation for odor blends; they do not establish that it outperforms simple pooling for the binary scent labels used here.

Hierarchical GNNs have also been used for property prediction within a single molecule, moving from atoms to fragments, as in HiGNN and HimGNN [18, 62]. POMMix, SolvGNN and GH-GNN,

in contrast, operate across molecules. The present study follows the second kind: a shared single-molecule encoder followed by an interaction step between the two molecules.

2.12 Olfactory Datasets and Label Reliability

This section distinguishes the single-molecule catalog labels from the pairwise blend labels, describes what each can and cannot support, and explains why label inconsistency, label noise, and label sparsity matter for evaluation.

2.12.1 Reference Catalogs

The quality of sources for public scent data are not equal. Community platforms such as *Fragrantica* and *Basenotes* describe finished commercial perfumes in the words of their users, which makes them unsuitable for learning how individual chemicals relate to scent [7, 55]. Because of these, researchers use catalogs such as *GoodScents* and *Leffingwell* for reference, as they record descriptors for individual aroma chemicals and have been used to train state-of-the-art prediction models [27, 44]. The *Leffingwell PMP 2001* database was compiled by olfactory experts, usually practicing perfumers, and its free-text descriptions were standardized into 113 descriptors by Sanchez-Lengeling et al.

2.12.2 The Blended-Pair Dataset

Sisson et al. [44] compiled blend labels from the lists of suggested blending partners on *GoodScents*, which group each partner under an odor heading. Their crawl was able to produce more than 160,000 pairs and, after standardization and filtering, resulted in up to 74 odor labels. Each molecule’s page was crawled separately, so a pair can be listed from both molecules’ pages; their pair-building code counts these listings but does not keep the count. The dataset has no concentration information, and its labels are catalog suggestions rather than measurements from a sensory panel. In this study they are therefore treated as catalog-recorded blend labels and used as proxy targets. Reciprocal listing and cross-catalog agreement are used as evidence of label consistency; they do not establish that a blend label is a correct description of human perception.

2.12.3 Human-Rated Datasets

Expert-compiled catalogs such as Leffingwell are assigned by people, but they are used here as training data and cannot serve as an independent check. Independent human-rated data are therefore needed. Human-rated scent datasets exist for single molecules and for mixtures, but to our knowledge none that gives scent descriptors for two-molecule blends is publicly available at the time of writing. Bierling et al. [6] provide yes-or-no ratings of 16 scent words for 74 single odors from about 1,200 laypeople. Pellegrino et al. [35] collected trained-panel ratings of mixtures with a 51-term list and have stated that their data will be released after publication [personal communication].

2.12.4 Absence of a Standard Vocabulary

No standard set of odor descriptors exists. Catalogs and studies use vocabularies of different sizes, and descriptors overlap in meaning [44]. To help structure this vocabulary problem, Sajan et al. [41] published an expert-curated taxonomy that groups raw odor words from the Pyrfume repository into 16 broader scent families. This kind of high-level grouping is useful for organizing descriptors across studies. This study maps its labels to these families for organized reporting only (Appendix B). The models still predict individual words, and no result or conclusion depends on the family mapping.

2.12.5 Noisy and Imbalanced Labels in Multi-Label Learning

Inconsistent labels act as noise in both the training targets and the test targets. Noise in the training targets can change what a model learns, because deep networks can memorize noisy labels [46]. Noise in the test targets can change which model appears best. Noise is harder to handle with several labels per example, because errors can be correlated across labels; Li et al. [30] model this by estimating a noise transition matrix that uses label correlations. For this reason, this study measures label consistency, evaluates on the most consistent labels, and tests whether the comparison between aggregation methods changes as label reliability decreases.

Multi-label scent data are also sparse: most descriptors are absent for most molecules, and some descriptors are far more common than others. Reviews of imbalanced multi-label classification describe imbalance within labels, among labels, and among label sets [50], and work on graph neural networks reports that the benefit of balancing techniques depends on the dataset and problem type

[2]. Because simple accuracy-style scores can look good on sparse labels, evaluation of such data needs metrics that are not dominated by true negatives [37].

2.13 Out-of-Distribution Splitting Strategies

This section will be the last technical segment of this chapter on data, providing a review of the standard data-splitting strategy of scaffold splitting, and an examination of how its application falls short in sparse chemical spaces, which then motivates the need for more rigorous out-of-distribution (OOD) evaluation. This section then grounds and discusses the clustering-based alternative our proposed framework will employ.

A common method to partition data for training, testing, and validation in chemoinformatics and molecular machine learning is scaffold splitting [16, 47]. In this method, molecules are represented by graphs of their structure, and the core framework of these graphs is referred to as their “scaffold” (Bemis–Murcko scaffolds) [57]. Molecules that share the same scaffold are grouped together, and these groups are then distributed among the training, testing, and validation sets to ensure that the test set contains structurally different molecules from the training set, which is a basic form of out-of-distribution (OOD) evaluation, which as opposed to random splitting, simply distributes molecules randomly across training, validation, and test sets [13, 16, 44, 47].

The caveat with this method is that, when the dataset is sparse, it collapses toward random splitting. For instance, similarly structured molecules get fragmented across both the train and test splits, leading to severe data leakage and inflated test scores as scaffolds in standard benchmarks could be populated by only one or two molecules each, undermining the grouping effect the method depends on [13]. Another problem demonstrated by scaffold splitting is that even molecules that are near-identical (as computed by Tanimoto similarity) [16, 47], even differing by as little as a single atom, can, under this method, end up in opposite splits, as observed by Guo et al. and Steshin et al. [16, 47], who argue that this problem causes models to “memorize” rather than generalize from the data. Because of these problems, scaffold splitting can misleadingly produce scores that are overly optimistic.

Shimakawa et al. [43] propose doing topological pre-clustering by projecting 2048-bit ECFP4 fingerprints onto a 2D map using 2D Uniform Manifold Approximation and Projection (UMAP), then grouping them together using spectral k -means clustering. This was then validated using Leave-

One-Cluster-Out Cross-Validation (LOCOCV), where clusters are left out in rotation for training and testing, such that what the model was trained on does not show up in what it is tested on, meaning the model actually learns, rather than simply memorizing the data it was trained on, which is important for OOD evaluation. More generally, leakage-reduced splitting can be formulated as an optimization problem over the data [24], and cluster-based splits have been reported to be among the most challenging for both classical and graph-based models [13].

In plain terms, one molecule can appear in many blend pairs, so splitting pairs at random can place the same molecule in both training and testing. Methods designed for single molecules do not address this, because an individual molecule can be a component of varying other molecules in different blends, meaning a molecule seen during training could reappear as a component in a different, unseen test blend, leaking information about that molecule across the split and inflating the performance metrics. Therefore, Sisson et al. [44] propose “graph carving”, using a meta-graph to describe the pairwise blend dataset, where each node represents an individual molecular graph and each edge represents a tested blend combination. Edges connecting a molecule from one partition to a molecule in another are discarded, so that no component is shared between splits. In their original implementation, this partitioning was done by randomly assigning molecules to each side and repeating until every label had at least one training and one test example, since a true scaffold-based split was not compatible with their sparse label distribution.

Building on this, our proposed framework instead applies the graph-carving protocol on top of a fingerprint-clustering partition (in the style of the UMAP and k -means approach discussed above), rather than random or scaffold-based partitioning. Clustering molecules and then carving the resulting blend meta-graph along those cluster boundaries yields a component-disjoint split. This reduces component and structural overlap between training and testing; it does not guarantee that every test molecule is entirely unlike every training molecule.

2.14 Evaluating and Comparing Models Under Label Sparsity

Sparse multi-label targets make some common scores misleading. Precision, recall, and F1 ignore true negatives and so expose a model that predicts no labels, whereas accuracy-style scores such as Hamming loss can look strong for such a model [37]. Threshold choice also matters: in a comparison

of graph and classical models for multi-label odor prediction, optimizing the decision threshold was reported to be essential [54].

Comparisons between models should also report how large a difference is, not only whether it is statistically significant. Bootstrap confidence intervals do this by showing the difference and its uncertainty together [5], and recent guidance cautions against naive paired t -tests when resampled test sets are used [37]. Together with the leakage problem above, this means that a fair comparison of aggregation methods needs matched encoders and training conditions, sparsity-aware metrics, validation-only threshold selection, and paired uncertainty estimates over the evaluated blends.

2.15 Synthesis and Research Gap

2.15.1 Synthesis of Literature

Current models in computational olfaction can successfully map single-molecule structures to their appropriate sensory descriptors [27]. However, because these compounds are evaluated in isolation, they do not address how two molecules should be combined, and catalog blend labels differ from the labels of the parent molecules in many blends [44]. Human-panel evidence on mixtures is mixed: continuous ratings are well predicted by a linear average of the components [35], while binary catalog labels record catalog-label loss and gain in many blends [44].

From an architectural standpoint, permutation-invariant set processing is needed to predict the behavior of multi-molecule mixtures [60, 28]. Simple linear pooling (mean or sum) can make different pairs look identical, because pairs with the same average produce the same combined representation [49].

Furthermore, standard scaffold splits suffer from severe data leakage at the component level in mixture spaces [13]. Clustering molecules and carving the blend meta-graph along cluster boundaries reduces component and structural overlap between training and testing [43, 44]. Finally, the blend labels in public data come from catalogs rather than sensory panels, and their reliability is rarely measured.

In summary: (1) a molecular graph encoder produces one vector per molecule; (2) the unresolved computing problem is how to combine the two vectors without making the prediction depend on input order; and (3) any such comparison must account for component leakage and uncertain labels.

2.15.2 Statement of the Research Gap

Existing work does not yet establish whether an attention-based set aggregator provides a reliable advantage over simpler alternatives when two molecular graph embeddings are combined for sparse, catalog-derived multi-label prediction. A fair answer requires the encoder and training conditions to be controlled, input order to be handled explicitly, repeated molecular components to be separated across partitions, and uncertain labels to be considered during evaluation. This study addresses that controlled evaluation gap.

2.15.3 Positioning of the Present Study

To address this gap, this study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation for combining the two molecule embeddings produced by a weight-shared graph attention encoder. The comparison is carried out on a component-disjoint, fingerprint-clustered and graph-carved partition of the data; the label-reliability analysis and the controlled-noise tests show how far the conclusions depend on the labels. A simpler baseline that matches or outperforms the Set Transformer would be a valid and informative result.

Chapter 3

Theoretical and Conceptual Framework

3.1 Theoretical Framework

This chapter follows the computation from the molecular input to the final evaluation. It first explains how a molecule is represented as a graph of atoms and bonds. It then describes how message passing and graph attention produce one embedding for each molecule. The next part explains why combining two embeddings is a separate design problem: a blend is an unordered pair, so the prediction should not depend on which molecule is listed first. The three aggregation methods are then described and compared. The chapter ends by explaining why sparse, catalog-derived labels affect both training and evaluation, and how reciprocal agreement, cross-catalog agreement, and controlled label corruption are used to study that problem. The discussion of averaging describes a possible information loss, not an observed result; the relative performance of the methods remains an empirical question.

3.1.1 Representing Molecules as Graphs

Each molecule is represented as an undirected, attributed graph. Atoms are the nodes, bonds are the edges, and the features attached to them become the inputs to the graph encoder [58]. This representation keeps the bond connectivity of the molecule instead of treating the structure as an

ordinary sequence of characters. SMILES strings are used to store and parse the structures, but different valid atom-numbering or starting-point conventions should describe the same molecular graph [34, 55].

Mathematical Formalism

Each molecule is defined as an undirected, attributed graph $G = (V, E)$, where V is the set of non-hydrogen atoms and E is the set of covalent bonds. In the representation used here, hydrogen information is included in the features of the corresponding heavy atoms rather than represented as separate nodes [4].

The connectivity can be written as an adjacency matrix A , where $A_{ij} = 1$ when atoms i and j are connected by a bond and $A_{ij} = 0$ otherwise [62]. The implementation stores this connectivity in a sparse form (Section 3.3.2). Each node and edge also has a feature vector:

- The node vector $\mathbf{x}_i \in \mathbb{R}^{d_x}$ records properties such as element, charge, valence, hybridization, aromaticity, hydrogen count, ring information, and chirality.
- The edge vector $\mathbf{e}_{ij} \in \mathbb{R}^{d_e}$ records bond properties such as bond type, conjugation, ring membership, and stereochemical configuration.

The complete node and edge feature tables are provided in Appendix A. RDKit supplies the parsed molecular structures and their features. The graph representation is not tied to a particular atom numbering: relabeling the nodes changes their order in the data structure, but not the bonds or the molecule they describe. This property is important because the message-passing and pooling operations should produce the same molecule representation when the atoms are listed in a different order. The final values of d_x and d_e are set by the approved feature specification in Chapter 4.

3.1.2 Graph Neural Networks and Message Passing

Most neural networks receive inputs in a fixed arrangement. A molecular graph does not have that form: molecules can contain different numbers of atoms, and their important relationships are represented by bonds. A Graph Neural Network (GNN) works with this structure directly. In the graph used here, a *node* represents an atom and its feature vector, while an *edge* represents a bond and its feature vector. The *neighborhood* $\mathcal{N}(i)$ of node v_i is the set of atoms directly connected to it by an edge [62, 4, 54].

The main operation in a GNN is *message passing*. During one layer, each node receives information from its neighborhood, combines those messages, and updates its own representation. For node v_i at layer t , one general form of this operation is:

$$\begin{aligned}\mathbf{m}_i^{(t)} &= \bigoplus_{j \in \mathcal{N}(i)} M_t(\mathbf{h}_i^{(t)}, \mathbf{h}_j^{(t)}, \mathbf{e}_{ij}), \\ \mathbf{h}_i^{(t+1)} &= U_t(\mathbf{h}_i^{(t)}, \mathbf{m}_i^{(t)}).\end{aligned}\tag{3.1}$$

Here, $\mathbf{h}_i^{(t)}$ is the current representation of node i , $\mathbf{e}_{ij}$ describes the bond to neighbor j , and M_t computes the message sent along that bond. The symbol $\bigoplus$ denotes the operation that combines all messages received by node i , while U_t produces its new representation. In ordinary terms, the node looks at itself, looks at its bonded neighbors, and then records an updated summary. Since the messages are combined as a group rather than in a prescribed order, changing the order in which neighboring atoms are stored does not change the information being combined. Repeating the layers allows information to travel from an atom to nearby atoms and then to larger parts of the molecule [18, 62].

After the final message-passing layer, a *graph readout* combines the node representations into one vector for the whole molecule. The readout specified for the final protocol is applied consistently to both molecules; when mean pooling is used, the node vectors are averaged before being passed to the blend-level models. This node-to-molecule step is common in molecular GNNs, although some recent models also learn representations for larger substructures [18, 54].

The GNN used here is a Graph Attention Network (GAT). Rather than assigning the same fixed weight to every neighboring node, a GAT learns weights from the node and bond features, allowing the encoder to give more influence to some local connections than to others. These weights are model parameters used to form a representation for descriptor prediction; they do not establish a receptor mechanism or a physical interaction between molecules [4, 54]. The specific attention calculation is described in the next subsection.

3.1.3 Graph Attention Encoder

The shared graph encoder maps each molecular graph to a fixed-length vector that can be used by any of the three blend-level aggregation methods. It learns representations of local atom and bond context that are useful for the descriptor prediction task. The same encoder is applied to the two molecules in a blend, so the resulting vectors are produced under the same learned rules [27, 58].

Let $\mathbf{h}_i^{(0)} = \mathbf{x}_i$ denote the initial feature vector of node v_i , with node-feature dimension d_x , and let $\mathbf{e}_{ij}$ denote the edge feature vector, with dimension d_e . These dimensions, as well as the hidden width and final embedding size, follow the approved architecture specification in Chapter 4; they are not copied from the earlier notebook. A learnable matrix projects the node states into the hidden space. To score how much node v_i should attend to neighbor v_j , a small learnable scoring function uses the projected states of both nodes and the edge features [58, 62]:

$$\tilde{e}_{ij} = \text{LeakyReLU}\left(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j \parallel \mathbf{W}_e\mathbf{e}_{ij}]\right), \quad (3.2)$$

where $\parallel$ denotes concatenation. Including $\mathbf{e}_{ij}$ lets the score use information about the bond connecting the two nodes. The activation and its parameters are part of the approved encoder specification [58].

Atoms have different numbers of neighbors, so the raw scores are normalized across the neighborhood $\mathcal{N}(i)$ of atom v_i with a softmax, so that the weights for each atom sum to one [58, 4]:

$$\alpha_{ij} = \text{Softmax}_j(\tilde{e}_{ij}) = \frac{\exp(\tilde{e}_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(\tilde{e}_{ik})}. \quad (3.3)$$

The normalized weights α_{ij} scale the projected neighbor features, and the selected node-update activation produces the updated representation of node v_i [58]:

$$\mathbf{h}_i^{(l+1)} = \sigma_{\text{upd}} \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j^{(l)} \right). \quad (3.4)$$

Here σ_{upd} denotes the node-update activation selected in the final encoder specification.

The encoder repeats this update for the number of layers specified in Chapter 4. If multiple attention heads are used, their outputs are combined according to that same specification. A graph readout then combines the final node states into one molecule vector; with mean pooling, this is the average of the final node representations. The readout produces the fixed-length vector $\mathbf{z}$ passed to the blend-level model. Thus, the node states retain local and progressively broader graph information, while the readout makes that information available as one vector for the molecule.

The three aggregation variants use a shared encoder initialization and matched training schedules, optimization settings, and budgets. The final encoder weights remain identical only if the approved protocol freezes the encoder. If the protocol fine-tunes the encoder end to end, the variants start from the same initialization but their final encoder weights may diverge during training. In that

case, the comparison is still controlled by the matched training policy, but it is not described as differing only in the aggregation operation.

The attention coefficients are learned computational weights used to form the representation. They indicate which neighboring features the model emphasizes for its prediction, but they do not demonstrate chemical importance, receptor activity, or a physical interaction between molecules.

3.1.4 Combining an Unordered Pair

The blend model receives two molecule embeddings, $\mathbf{z}_A$ and $\mathbf{z}_B$. Because the pair has no meaningful first or second member, the model should return the same prediction for (A, B) and (B, A) . In other words, changing the order in which the two molecules are supplied must not change the predicted descriptor vector [44, 60].

The three aggregation methods satisfy this requirement in different ways. Their pair representations and main trade-offs are summarized below.

Table 3.1: Aggregation methods for an unordered pair of molecule embeddings.

Method	Pair representation	How order is handled	Main trade-off
Mean pooling	Average the two embeddings	Invariant by construction	Can discard pair-specific distinctions
Order-averaged concatenation	Evaluate both concatenation orders and average their predictions	Explicitly symmetrized	Requires two ordered evaluations
Set Transformer aggregation	Let the two embeddings exchange learned information before pooling	Invariant set processing	Adds parameters and computation

Mean Pooling

Mean pooling is the simplest order-independent baseline. It represents the pair by the element-wise average

$$\mathbf{z}_{\text{mean}} = \frac{\mathbf{z}_A + \mathbf{z}_B}{2}. \quad (3.5)$$

Swapping the two inputs gives the same vector automatically, so this method is invariant by construction. Its limitation is that different pairs can share the same average. For example, the one-dimensional pairs $(2, 8)$ and $(4, 6)$ both produce 5. This possible loss of pair-specific distinction

motivates the comparison with the other two methods; it is not a claim about the observed results [60, 49].

Order-Averaged Concatenation

Concatenation keeps the two embeddings side by side, but one ordered concatenation depends on which molecule is listed first. The adapted baseline therefore evaluates both $[\mathbf{z}_A | \mathbf{z}_B]$ and $[\mathbf{z}_B | \mathbf{z}_A]$, then averages the two resulting model outputs. This explicit symmetrization makes the final prediction the same for both input orders while retaining more pair-specific information than a simple average. It requires two ordered evaluations for each pair.

The final protocol must state whether the two outputs are averaged as logits before the sigmoid or as probabilities after the sigmoid. That choice will be made when the team finalizes the training and evaluation specification and will be used consistently in the methodology, implementation, and reporting. This is an adapted baseline based on the pairwise concatenation design of Sisson et al., not a reproduction of their complete model [44].

Set Transformers and Attention-Based Pooling

The Set Transformer receives the two embeddings as a set $X = \{\mathbf{z}_A, \mathbf{z}_B\}$. Its attention blocks allow the embeddings to exchange learned information before a pooling step produces one blend representation [28, 51]. The Self-Attention Block (SAB) processes the two members without assigning them a permanent first or second position, so the resulting set representation is unchanged when the inputs are swapped. The method can represent statistical relationships between the learned embeddings that mean pooling may discard, but it adds parameters and computation.

The second block is Pooling by Multihead Attention (PMA):

$$\text{PMA}_k(X) = \text{MAB}(S, \text{rFF}(X)). \quad (3.6)$$

This process begins by passing the molecular embeddings $\mathbf{X}$ through a row-wise feedforward transformation. The PMA block uses learned seed vectors S to attend to these transformed embeddings and compress them into a fixed-size blend representation [28, 44]. In this study, the attention weights are interpreted as computational parameters for combining representations. They model statistical relationships between embeddings for descriptor prediction, not physical molecular interactions or

receptor mechanisms. Whether the additional capacity improves prediction is tested empirically under the same training and evaluation conditions as the baselines.

3.1.5 Representation Trade-offs

The three methods make different compromises when two molecule embeddings are reduced to one blend representation. Mean pooling is inexpensive and order independent, but different pairs can have the same average, so some pair-specific distinctions may be lost [60, 49]. It remains a credible baseline because its simplicity can be sufficient when the averaged embedding retains the information needed for the descriptor task. The order-averaged concatenation baseline retains the two embeddings side by side and restores order invariance by evaluating both input orders, at the cost of a second ordered evaluation. Set Transformer aggregation has additional learned capacity and computation, and uses that capacity to model statistical relationships between the embeddings before pooling. These are representational trade-offs, not assumptions about the chemistry of a blend.

Why Averaging Can Lose Information

Averaging two molecule embeddings can make structurally different pairs indistinguishable to the model. In one dimension, the pairs (2, 8) and (5, 5) both average to 5, so the averaged representation cannot distinguish those pairs. This example demonstrates a possible information loss; it does not show that such collisions are common or dominant in the present dataset. The actual usefulness of mean pooling remains an empirical question. The order-averaged concatenation baseline retains the two embeddings side by side while correcting for the two possible input orders, and the Set Transformer provides another order-invariant way to combine them by learning statistical relationships before pooling. A full derivation of the symmetry argument is given in Appendix A.1.

3.1.6 Sparse Labels and Label Reliability

The output is a sparse multi-label vector: most blend-descriptor cells are absent, while a smaller number are recorded as present. This imbalance affects both parts of the experiment. During training, the many easy negative cells can dominate the loss. During evaluation, a model that predicts mostly negatives can look favorable on an aggregate score even when it misses useful positive descrip-

tors. The study therefore uses metrics suited to sparse multi-label outputs and applies a reliability mask when only the more consistent cells are being evaluated. The exact mask construction and training conditions are specified in Chapter 4.

Because the blend labels used in this study are compiled from catalog recommendations rather than a controlled sensory panel, their reliability must be considered before model comparisons can be interpreted. The reliability checks describe the strength and consistency of the catalog evidence; they do not establish perceptual truth.

Reciprocal agreement is a within-catalog check. In the GoodScents catalog, molecule A’s page may list molecule B as a blending partner under a given scent word, but molecule B’s page may not list molecule A under the same word. When both pages agree, the label is *reciprocal*; when only one page makes the claim, the evidence is weaker. Reciprocal evidence therefore strengthens confidence in a recorded positive label, but it does not prove that the descriptor is perceptually correct.

Cross-catalog agreement is measured using Cohen’s kappa (κ), which quantifies how consistently two independently maintained catalogs (GoodScents and Leffingwell) assign the same descriptor to the same molecule, correcting for the agreement expected by chance alone. A commonly used benchmark scale classifies κ values between 0.21 and 0.40 as “fair” agreement, 0.41 to 0.60 as “moderate,” and above 0.60 as “substantial” agreement [52]. These categories are used as operational descriptions of descriptor-level consistency, not as universal or objective judgments of label correctness. The thresholds and the catalogue records used to compute them are specified in Chapter 4.

The target representation follows a closed-world assumption for the purpose of the supervised task: an unlisted descriptor is encoded as a negative target. This is a modeling convention, not a claim that the odor is truly absent; catalogs can omit descriptors for reasons unrelated to sensory absence.

Reliability is applied at the blend–descriptor cell level. A Gold-only comparison must include eligible negative cells as well as reliable positive cells. Otherwise, a test set containing only recorded positives could not penalize false-positive predictions, making measures such as F1 or PR-AUC misleading. Cells whose evidence is ambiguous are masked from the relevant loss or metric rather than causing the entire blend to be discarded. Silver and Bronze labels indicate weaker evidence for the comparison; they do not prove that a label is wrong.

Even after reliability screening, some label noise may remain. The controlled label-corruption experiment described in Chapter 4 tests how the aggregation methods respond as training-label

reliability is reduced. A smaller performance decline under the same corruption condition would indicate greater robustness to that tested form of noise, but it would not establish that the labels are perceptually accurate [46].

3.2 Conceptual Framework

The conceptual framework of this study operationalizes the theoretical formulations into a systematic computational pipeline structured around the Input-Process-Output (IPO) model. Figure 3.1 outlines the structural interaction between input data representations, the two-level hierarchical deep learning architecture, and the downstream analytical and practical deliverables.

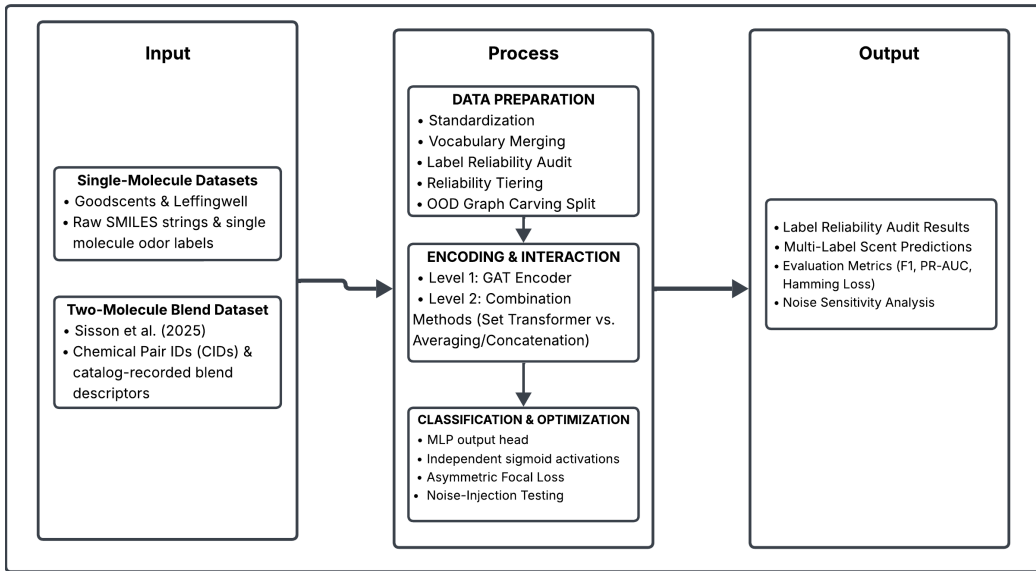


Figure 3.1: Input-Process-Output (IPO) Model.

3.2.1 Input Phase

The computational data pipeline starts with the acquisition, curation, and gathering of data from olfactory databases. The raw SMILES strings are extracted from the GoodScents and Leffingwell repositories accessed via the Pyrfume Library in Python [7]. These raw chemical blueprints are then paired with catalog-recorded odor tags to establish baseline inputs required from the pre-training

phases. The binary mixture profiles are retrieved from the Sisson et al. (2025) benchmark dataset [44]. The retrieved benchmark dataset provides standardized chemical pair identifiers (CIDs) and multi-label scent descriptors that will help the network to learn and predict the combined scent of two scent blends.

3.2.2 Process Phase 1: Data Preparation and Graph Carving

To raw chemical input will first pass through a standardization pipeline. Salts and other non-fragrance components are removed using RDKit’s SaltRemover module, leaving the main molecule. Compounds containing elements other than C, H, O, N, S, F, Cl, Br, and I are dropped, since these fall outside the range typical of aroma chemicals [34]. The molecular weight should be between 20 and 600 g/mol. Below this range is considered irrelevant, while above the range is considered too heavy to be perceived as a scent at all [27]. The surviving SMILES strings are canonicalized using RDKit. The scent descriptors text from the source databases is standardized by merging redundant terms and eliminating descriptors that appear rarely to learn from [9]. The blend labels are then audited for reliability before any model is trained. Reciprocal agreement is measured for each blend pair, and cross-catalog agreement between GoodScents and Leffingwell is measured on the parent molecules using Cohen’s kappa. The resulting evidence is used to construct the cell-level reliability conditions described in Chapter 4. Gold, Silver, and Bronze, when used, describe stronger or weaker evidence for evaluation; they do not prove that a label is true or false. Unlisted descriptors are treated as negative targets under the study’s closed-world assumption, while ambiguous cells are masked rather than used to remove an entire blend. To test how well the network handles unfamiliar compounds, molecules are grouped into structural families by UMAP and K-Means on 2048-bit Morgan fingerprints [16]. The Graph-Carving algorithm then divides the mixture meta-graph along family clusters. This guarantees that validation and testing pairs do not contain any molecule encountered during training [13].

3.2.3 Process Phase 2: Encoding and Pair Representation

The second phase converts each molecular graph into a fixed-length vector. Atoms and bonds are processed as graph nodes and edges, and the shared GAT encoder updates the node representations before applying the approved graph readout [58]. The same initialization and encoder policy are used for both molecules, with the final feature dimensions, layer and head counts, hidden widths,

and embedding size taken from the approved Chapter 4 specification rather than copied from the earlier notebook.

The two molecule embeddings are then combined into a single blend representation using one of three aggregation methods: mean pooling, order-averaged concatenation following the adapted pairwise baseline of Sisson et al. [44], or Set Transformer aggregation. These methods are compared under matched training schedules. The Set Transformer is treated as a learned way to combine the two representations; the study does not claim that its attention weights describe physical or receptor-level interactions.

3.2.4 Process Phase 3: Classification and Optimization

Once the aggregation module yields the blend representation, it is translated into multi-label descriptor predictions. The Multi-Layer Perceptron (MLP) head maps the approved blend-embedding size through the approved hidden width and ends with L individual outputs to match the target scent classes [44]. Because scent profiles are inherently multi-label, the classification layer utilizes independent sigmoid activation functions [40]. These activations compute individual probability scores for the joint presence or absence of each unique target scent descriptor. The encoder policy is finalized in Chapter 4. If the approved protocol uses end-to-end fine-tuning, prediction errors update both the shared molecular encoder and the aggregation module; the final encoder weights may then diverge across variants even though initialization and training schedules are matched. If the encoder is frozen, only the aggregation and output modules are updated. The loss and its imbalance parameters follow the approved Chapter 4 protocol [19]. Each combination method is trained under the reliability conditions specified in Chapter 4, including a condition that uses eligible high-reliability cells and their eligible negative cells. The noise-sensitivity test applies the declared controlled label corruption to the same matched conditions. Olfactory datasets are highly sparse, meaning a single scent profile contains only a few active descriptors alongside dozens of inactive ones (zeros). Under standard training rules, the model could appear accurate simply by predicting “no” for most labels. The asymmetric loss addresses this class imbalance by reducing the influence of obvious negatives and forcing the network’s mathematical focus onto the rare, active scent notes that are difficult to predict [19].

3.2.5 Output Phase

The final phase outputs a multi-label binary vector of predictions across all L target scent descriptors which is based on the probability given to each descriptor. A validation module evaluates the model’s performance on these predictions under the declared reliability masks. It calculates metrics such as macro- and micro-averaged F1-scores, Hamming loss, and the Precision-Recall Area Under the Curve (PR-AUC) [3, 50, 54]. The reliability-aware comparison includes eligible negative cells so that false-positive predictions remain evaluable; ambiguous cells are excluded by the mask. The exact partitions, masks, and statistical comparison procedure are specified in Chapter 4.

The study additionally reports the results of the label reliability audit, including reciprocal agreement rates and cross-catalog agreement by descriptor.

3.3 Technical Background

Mapping molecular structure onto multi-label olfactory predictions requires stitching together a few different layers of infrastructure: cheminformatics standards to make sure the molecules are valid, a graph learning framework to process them, and a loss function suited to how sparse the labels are. This section walks through each piece—the parsing standards, the sparse graph representation, and the loss objective—and explains why each choice was necessary rather than incidental.

3.3.1 Cheminformatics Standard (RDKit): SMILES Parsing and Sanitization

Molecular structures in this study are represented as Simplified Molecular Input Line Entry System (SMILES) strings, a compact text-based notation, which are canonicalized and parsed into undirected, attributed molecular graphs using the open-source RDKit library in Python [34]. SMILES strings are convenient because they are compact and suitable for storage. This compactness comes at a cost, however: they are structurally sensitive, prone to syntax errors, and do not encode explicit 3D geometry [55]. Before any structure reaches the model, every string passes through RDKit’s sanitization pipeline, which runs four checks in sequence:

- Kekulization checks that the alternating single and double bonds within conjugated or aromatic rings are represented in a chemically consistent manner, resolving ambiguous resonance

structures.

- Aromaticity Perception evaluates the coordinate and topological structure of each atom to identify aromatic rings, dynamically reassigning aromatic bond attributes to relevant edges.
- Valence Checking checks every atom, specifically heavy atoms—carbon, nitrogen, oxygen, sulfur, and halogen to ensure it does not exceed its physical valency limits. This rejects syntactically corrupted SMILES strings or physically impossible hypervalent structures.
- Ring Perception, which identifies and indexes ring systems, information the later convolutions depend on.

Enforcing these rules ensures that what the network receives is already known to be chemically valid, rather than requiring the model to infer the rules of organic valency implicitly from training data [4, 62]. The sanitization pipeline acts as an explicit structural filter ahead of the model, so that the Level 1 Graph Attention Network (GAT) operates only on topologically valid graphs that correspond to real molecular structures [27, 58].

3.3.2 Deep Learning Stack (PyTorch / PyTorch Geometric):

Sparse Graph Convolutions

After sanitization, each molecule is formatted as an undirected graph $G = (V, E)$, where atoms are nodes (V) and covalent bonds are edges (E). A dense $N \times N$ adjacency matrix could represent this connectivity, but for molecules, where most atoms bond to only a few neighbors, this wastes memory, scaling at $\mathcal{O}(N^2)$ for structures that are inherently sparse. The architecture instead runs on PyTorch Geometric, which handles molecular graphs as sparse attributed objects rather than dense arrays [12].

Topology is stored in Coordinate (COO) format: a sparse two-row index matrix C of size $2 \times 2E$, where the first row lists source node indices and the second lists receiver node indices [4]. Since chemical bonds are physical and undirected, every source node is also a receiver, so $2E$ entries are needed rather than E — each bond is represented in both directions so that message-passing can update both of its atoms. Node states are updated by running the GAT computation described in Section 3.1.3 over this sparse index matrix, rather than over a dense adjacency matrix, at each layer l .

3.3.3 Multi-Label Mathematical Formulation:

Independent Sigmoids and Sparsity Mitigation

A molecule or blend can carry several scent descriptors at once (for example, both fruity and green), rather than belonging to a single exclusive category. The descriptors can therefore co-occur, and predicting one descriptor should not force all other descriptors to be absent. The final layer skips a mutually exclusive softmax and instead maps the approved blend embedding through an MLP to L independent sigmoid outputs, one per target descriptor [40]. Each output is a probability $\hat{y}_{ik}$ between 0 and 1 for example i and descriptor k :

$$\hat{y}_{ik} = \sigma(s_{ik}) = \frac{1}{1 + e^{-s_{ik}}}. \quad (3.7)$$

Here s_{ik} is the output score before the sigmoid. The target y_{ik} is binary for each descriptor, so the model makes one present/absent decision per descriptor rather than one choice from a mutually exclusive list.

A multi-label dataset is sparse when the number of positive cells is small relative to the total number of example-descriptor cells. Let N be the number of examples and L the target-vocabulary size. The label density can be described without fixing a provisional value as $\rho = (NL)^{-1} \sum_{i=1}^N \sum_{k=1}^L y_{ik}$. A small ρ means that most cells are negative. The many easy negatives can then dominate a plain loss, allowing a model that predicts mostly zeros to appear successful while missing the positive descriptors that matter for the comparison.

A standard multi-label setup would optimize independent Binary Cross-Entropy terms across the L descriptors:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{NL} \sum_{i=1}^N \sum_{k=1}^L \left[y_{ik} \log(\hat{y}_{ik}) + (1 - y_{ik}) \log(1 - \hat{y}_{ik}) \right]. \quad (3.8)$$

The planned loss is an Asymmetric Focal Loss. It separates the focusing of positive and negative terms through exponents γ_p and γ_n and uses a class-frequency weight α_k [19]:

$$\mathcal{L}_{\text{FL}} = -\frac{1}{NL} \sum_{i=1}^N \sum_{k=1}^L \left[y_{ik} \alpha_k (1 - \hat{y}_{ik})^{\gamma_p} \log(\hat{y}_{ik}) + (1 - y_{ik}) (1 - \alpha_k) \hat{y}_{ik}^{\gamma_n} \log(1 - \hat{y}_{ik}) \right]. \quad (3.9)$$

The exponents and class weights determine how strongly easy negative terms are down-weighted relative to rarer positive terms. Their final values are part of the approved Chapter 4 training specification, rather than provisional values copied from the earlier notebook. The purpose of the

loss is to reduce the effect of the imbalance; it does not remove the need for reliability masks or appropriate evaluation metrics.

3.3.4 Evaluation Metrics for Sparse Multi-Label Outputs

Macro F1 is the primary metric for the comparison. It computes F1 separately for each evaluated descriptor and then averages those values, so frequent descriptors do not determine the result by themselves. This is useful when the study compares performance across a vocabulary whose descriptor frequencies may differ.

Micro F1 is reported as a supporting metric by pooling true positives, false positives, and false negatives across descriptors before calculating one F1 value. PR-AUC is also supporting: it measures how well the model ranks positive cells across decision thresholds, which is informative when positives are rare. Hamming loss reports the fraction of evaluated cells classified incorrectly, so it gives a complementary view of overall cell-level errors. The metric definitions and masking rules are specified in Chapter 4.

Hamming loss must be interpreted against an all-negative reference because a sparse label matrix can make an always-negative predictor appear strong. Let M_{ik} be the evaluation mask and define $\hat{y}_{ik}^{(0)} = 0$ for every eligible cell. The corresponding reference is

$$\text{HL}_{\text{neg}} = \frac{\sum_{i=1}^N \sum_{k=1}^L M_{ik} y_{ik}}{\sum_{i=1}^N \sum_{k=1}^L M_{ik}}. \quad (3.10)$$

This is the error made by predicting negative for every evaluated cell. A model's Hamming loss should therefore be reported with this reference rather than interpreted in isolation. The reference is computed separately for each declared evaluation mask.

Chapter 4

Methodology

This chapter describes the preparation of the catalog-derived datasets, the construction of reliability masks, the formation of component-disjoint data partitions, and the matched training and evaluation of three aggregation methods. It also defines the robustness conditions, metrics, and paired statistical comparisons used to answer the research questions. Because this is a proposal, the procedures are written in the future tense. Settings that have not yet been approved, and numbers that can only be known after the final pipeline is run, are shown as symbols or marked as pending; none of them are results.

4.1 Research Design

This study will use a quantitative experimental design on a multi-label molecular graph classification task. It compares three ways of combining the learned embeddings of the two molecules in a blend: mean pooling, order-averaged concatenation, and Set Transformer aggregation. Fragrance-blend prediction is the application. The computer-science investigation concerns how an unordered pair of graph embeddings should be aggregated, how uncertain labels should be handled, and how models should be evaluated when the two molecules of a pair may also appear in other pairs.

The research questions are the following.

RQ1. Under matched encoding, training, and evaluation conditions, how does the performance of Set Transformer aggregation compare with mean pooling and order-averaged concatenation on

the reliability-screened test data?

RQ2. How does the performance of Set Transformer aggregation relative to the two baselines vary across additive, catalog-label loss, and catalog-label gain patterns?

RQ3. How does the robustness of Set Transformer aggregation compare with the two baselines as training-label reliability decreases, including under controlled label corruption?

Table 4.1 states what is varied and what is held fixed. The aggregation method is the primary varied factor. Label-reliability and label-corruption conditions are robustness factors, and the catalog-label pattern of a blend is used only for a secondary subgroup analysis. Everything else is a matched control, so that a difference between aggregation methods is not caused by a difference in data, training budget, or luck of the seed.

Table 4.1: Varied factors and matched controls.

Category	Treatment
Aggregation method	Primary varied factor (three levels)
Label-reliability and corruption conditions	Robustness factors
Catalog-label pattern	Secondary subgroup analysis
Encoder design and initialization policy	Matched control
Train/validation/test partitions	Fixed across variants
Loss, optimizer, stopping rule, training budget	Matched controls
Random seeds and corrupted datasets	Paired across variants

The dependent variables are macro F1 (the primary metric, declared before any experiment is run), micro F1, PR-AUC, and Hamming loss, defined in Section 4.7.

4.2 Data Acquisition and Preparation

This section describes the data sources, how they are joined, and how the model inputs (molecular structures) and the model targets (odor descriptor labels) are prepared. The two are prepared separately: structures become graphs, and descriptors become a fixed list of labels.

4.2.1 Data Sources and Provenance

The study will use three sources. The single-molecule sources are GoodScents and Leffingwell, both accessed through Pyrfume. They are independently maintained reference catalogs for the perfumery and flavor industries. Each record lists a molecule, its structure, and the scent descriptors assigned to it (for example, fruity or sweet). The Leffingwell descriptors are described as assigned by olfactory experts, usually practicing perfumers [27]. The blend source is the two-molecule dataset of Sisson et al. [44], obtained from their public repository.

Each Pyrfume source has three files: `behavior.csv`, which holds stimulus–descriptor pairs; `stimuli.csv`, which holds the stimulus identifier, the GoodScents or Leffingwell identifier, the PubChem CID, concentration, and solvent; and `molecules.csv`, which holds the CID, molecular weight, isomeric SMILES, IUPAC name, and name. Descriptors in `behavior.csv` are indexed only by stimulus, so they will first be joined to `stimuli.csv` on the stimulus field to obtain a CID, and then to `molecules.csv` on the CID. Concentration and solvent columns will be dropped, because concentration is outside the scope of this study.

Structure filtering. Before molecules are matched across sources, structures will be sanitized with RDKit: salts are removed, molecules containing elements other than C, H, O, N, S, F, Cl, Br, and I are excluded, and molecules outside a molecular-weight range are excluded. **[PENDING: confirm the element list and the molecular-weight range (20–600 g/mol in the earlier pipeline) and the resulting molecule count, which is a result of the final run and is not stated here.]** This step uses structure only; no label is consulted.

Join key. The two single-molecule catalogs will be merged into one master dictionary using the isomeric SMILES string as the key. Every SMILES string in both sources, and in the blend dataset, will be canonicalized with RDKit so that the same molecule written in two different ways is matched correctly. SMILES was chosen over the PubChem CID because some CIDs in the source files are negative or non-standard and would need further standardization, whereas SMILES must be canonicalized anyway when molecules are converted to graphs. Blend components in the Sisson et al. dataset, which are identified by CID, will be linked to the master dictionary by canonicalizing their SMILES in the same way.

Blend records. The blend dataset of Sisson et al. was built by crawling the “Potential Blenders” section of each GoodScents molecule page, which lists the molecules that molecule could be paired with under scent headings. The labels of a pair are the headings under which the partner was listed. **[CONFIRM: re-read the Sisson et al. Methods and supplement and state exactly how a heading becomes a blend label before this sentence is final.]** Because each molecule’s page was crawled separately, a pair can be listed from both molecules’ pages. The pair-building code of Sisson et al. counts how many pages listed a pair under each heading (one or two) but does not keep the count. This study will keep that count, because it provides the reciprocal evidence used in Section 4.3.

4.2.2 Ground Truth and Its Limits

Catalog-recorded descriptors serve as proxy targets for supervised learning. Reliability procedures estimate the consistency of those records; they do not establish perceptual correctness. The catalogs are curated reference records, not live sensory panel measurements, and their descriptors come from predefined word lists without a reported procedure for combining individual judgments. They therefore contain some noise and subjectivity [6]. This study treats them as a practical proxy for perceived scent, estimates how consistent they are, and examines how model comparisons change as label reliability changes.

For the computational scope of this study, each blend is treated as a simple physical mixture of two aroma chemicals. No significant intermolecular reaction is expected for most pairs under the assumed blending conditions (room temperature, no catalyst), so the data are treated as standard fragrance blending rather than reaction chemistry. A few highly reactive pairs may deviate from this expectation. **[CONFIRM: wording comes from chemistry faculty; permission to name the source is pending.]**

4.2.3 Target Vocabulary

Two vocabularies must be kept apart. The *standardized vocabulary* is the list of descriptor names after spelling variants have been merged. The *final target vocabulary*, of size L , is the subset of the standardized vocabulary on which models are actually trained and scored.

Standardization. The raw GoodScents and Leffingwell labels contain spelling variants of the same descriptor (for example, anisic and anise, wine and winey, or caramel and caramelic). Reviewing every label by hand would be impractical, so the study will adopt the open-source canonicalization dictionary of Sisson et al. [44], which merges raw blend descriptors into 104 standardized descriptors. **[CONFIRM: the source text gives both “approximately 400 raw descriptors” and “109 raw notes” for the input to this step; check which is correct and use one.]** Any descriptor in the merged data that the dictionary does not cover will be reviewed manually and either mapped to an existing descriptor or declared new, followed by whitespace and punctuation normalization. The standardized single-molecule vocabulary may exceed 104 descriptors if new descriptors survive this review; Appendix B will list it.

Final target vocabulary. The final target vocabulary will be built from *training data only* (Section 4.5.4). Validation and test labels will not be consulted when deciding which descriptors are kept. The size L is therefore not known yet and is written as a symbol throughout. The figure 104 is the size of the standardized vocabulary before partitioning, and the figure 74 is the number Sisson et al. reported after their own partitioning; neither is assumed to be this study’s L .

Inputs and targets. Input preparation (canonical SMILES to molecular graph with atom and bond features, Section 4.4.1) uses only molecular structure. Target preparation (descriptor standardization, mask construction, vocabulary selection) uses only descriptor labels. Neither step uses the test partition to make a decision.

4.2.4 Label Families (Reporting Only)

Expert-curated odor taxonomies, such as that of Sajan et al. [41], group raw odor words into broader families (for example, rose, jasmine, and violet under floral). The model will still predict each descriptor individually, because catalog-label loss and gain refer to specific words rather than to families. The Sajan et al. grouping is used for organized reporting only: Appendix B will map each descriptor to a family and give the number of descriptors per family. No per-family scores will be computed.

4.3 Label Reliability Mask

Not every recorded label is equally trustworthy, and not every missing label is a true absence. Rather than assigning one reliability grade to a whole blend, the study will decide, for each *blend-descriptor cell*, whether that cell is eligible for a stricter, “Gold-only” treatment. A cell is one entry of the blend-by-descriptor label matrix: one blend and one descriptor.

4.3.1 Consistency Evidence

Two kinds of evidence are used. Both measure consistency, not correctness.

- **Reciprocal-positive evidence (cell level).** A recorded positive is *reciprocal* if the pair was listed under that descriptor from the pages of both molecules (a count of two). Using the pair-building code of Sisson et al. [44], a preliminary audit found that 61,838 of 235,040 listings (26.3%) were reciprocal. **[CONFIRM: this is a preliminary audit figure; recompute with the final pipeline before reporting.]** A one-sided listing is not necessarily wrong, since catalogs do not list every possible blend, but it is not independently confirmed.
- **Descriptor-level consistency (descriptor level).** Agreement between GoodScents and Leffingwell on the molecules they share will be measured separately for each descriptor using Cohen’s kappa. In a preliminary audit on 2,225 shared molecules, 64 of 104 descriptors reached a kappa of at least 0.21 (“fair” on the scale of Landis and Koch). **[PENDING: the statistic and the cutoff are not final. The team must approve the statistic, the cutoff, and the molecule set used to compute it.]** A descriptor that meets the cutoff is called *consistent*. Reaching the cutoff does not prove that the labels are correct.

4.3.2 Eligibility Mask

Table 4.2 defines the eligibility mask $M_{ij} \in \{0, 1\}$ for blend i and descriptor j in the Gold-only treatment.

Because eligible unlisted descriptors enter as negatives, a Gold-only test set is not positive-only: a model that predicts a descriptor that is not recorded for a blend is still penalized as a false positive.

Closed-world assumption. Treating an unlisted descriptor as a negative is an operational assumption. It is needed because supervised multi-label learning requires a definition of “absent.”

Table 4.2: Eligibility of a blend–descriptor cell in the Gold-only treatment.

Cell evidence	Descriptor is consistent	Gold-only treatment
Reciprocal recorded positive	Yes	Include as positive ($M_{ij} = 1$)
One-sided or otherwise ambiguous recorded positive	Either	Mask ($M_{ij} = 0$)
Unlisted descriptor	Yes	Include as negative ($M_{ij} = 1$), under the closed-world assumption
Any cell	No	Mask ($M_{ij} = 0$)

Catalog omission may not mean true sensory absence, since a catalog may simply not list a blend or a descriptor. The reliability mask reduces this uncertainty but does not remove it, and results will be interpreted with this limitation in mind.

4.3.3 Experimental Label Conditions

Two label conditions are used.

- **Full-label condition.** Every cell is used ($M_{ij} = 1$ for all i, j), with listed descriptors as positives and unlisted descriptors as negatives.
- **Gold-only condition.** Only cells with $M_{ij} = 1$ under Table 4.2 are used.

Masked loss. In the Gold-only condition a blend is kept as long as it has at least one eligible cell; it is not removed merely because some of its cells are ambiguous. Loss is computed only over eligible cells:

$$\mathcal{L}_{\text{masked}} = \frac{\sum_i \sum_j M_{ij} \ell_{ij}}{\sum_i \sum_j M_{ij}} \quad (4.1)$$

where ℓ_{ij} is the per-cell loss (Section 4.4.3). In the full-label condition $M_{ij} = 1$ everywhere and the expression reduces to the ordinary mean loss. The same mask is applied when Gold-only metrics are computed (Section 4.7). The training, validation, and test partitions each carry their own mask, built from the same rule.

Insufficient support. After masking, a descriptor may have no eligible positives, or no eligible negatives, in a partition. F1 and PR-AUC are undefined for such a descriptor. Any descriptor with fewer than s eligible positives or fewer than s eligible negatives in the evaluated partition will

be excluded from macro-averaged scores, and the excluded descriptors will be listed in the report. **[PENDING: the minimum support s must be fixed before final evaluation.]** A training run that has no eligible cells at all is reported as a failed run and is not silently dropped.

4.4 Model Architecture

All three models share one molecular graph encoder and one output head. They differ in the step that combines the two molecule embeddings. The data flow is: each molecule is turned into a graph and encoded into a vector; the two vectors are combined by the aggregation method; and the combined vector is converted into one probability per descriptor.

4.4.1 Shared Graph Encoder

Each molecule is represented as a graph in which atoms are nodes and bonds are edges. A Graph Attention Network (GAT) encoder turns each graph into a fixed-length vector $\mathbf{z} \in \mathbb{R}^d$, so a blend yields two embeddings $\mathbf{z}_A$ and $\mathbf{z}_B$. The same encoder, with the same weights, is applied to both molecules.

[CONFIRM: node feature size, edge feature size, number of GAT layers, attention heads, readout, and embedding size d must be copied from Chapter 3 after the revised architecture is approved. The values used in the earlier notebook (46 node features, 10 edge features, 3 layers, $d = 256$) are not inherited automatically.] The attention computation is defined in Chapter 3 **[TODO: replace with \ref to the Chapter 3 section label]**. In the rest of this chapter d stands for the embedding size and, where a concrete number is needed for illustration, $d = 256$ is used.

Pretraining. The encoder may first be pretrained on single-molecule data, using a temporary head with one sigmoid output per standardized single-molecule descriptor and binary cross-entropy loss. Only molecules assigned to the training partition (Section 4.5) will be used, so molecules reserved for validation and testing are never seen during pretraining. After pretraining the temporary head is discarded and only the encoder weights are kept.

Encoder policy. **[PENDING: the team must choose one policy for the final comparison.]** The earlier notebook’s freeze/unfreeze schedule is implementation experience only and is not

the revised protocol. The two options are:

- a *frozen shared encoder*, in which encoder weights are fixed after pretraining and only the aggregation module and output head are trained; or
- the *same pretrained initialization followed by matched end-to-end fine-tuning* for all variants.

If the second option is chosen, the text will state: “The variants begin from the same encoder initialization and use matched schedules and optimization settings. Their encoder weights may diverge during end-to-end training.” In that case this study will not claim that the final models differ only in how they combine embeddings, because their encoders can end up different. Either way, the initialization policy, schedule, and training budget will be identical across the three aggregation methods.

4.4.2 Aggregation Variants

Table 4.3 summarizes the three variants. For two molecules, the embeddings form an unordered pair $\{\mathbf{z}_A, \mathbf{z}_B\}$, and a blend prediction should not change if the two molecules are swapped (permutation invariance).

Table 4.3: The three aggregation variants.

Variant	General operation	Order handling
Mean pooling	Average the two molecule embeddings	Invariant by construction
Order-averaged concatenation	Run both ordered concatenations through the head and average the outputs	Explicit symmetrization
Set Transformer aggregation	Apply set attention, then invariant pooling	Invariant set processing

Mean pooling. The blend embedding is the unweighted average of the two molecule embeddings,

$$\mathbf{z}_{\text{blend}} = \frac{1}{2}(\mathbf{z}_A + \mathbf{z}_B) \in \mathbb{R}^d. \quad (4.2)$$

This is simple and invariant to order. Sum pooling is not included as a separate variant: for exactly two molecules the sum is twice the mean, so it adds no information once the output head is trainable.

Order-averaged concatenation. The two embeddings are placed side by side, $[\mathbf{z}_A | \mathbf{z}_B] \in \mathbb{R}^{2d}$ (512 dimensions when $d = 256$). Raw concatenation depends on the order of its inputs, so the model is evaluated on both (A, B) and (B, A) and the two outputs are averaged. This follows the pairwise design of Sisson et al. [44], but it is an *adapted baseline*, not a reproduction of their whole model: it uses this study’s encoder, loss, and partitions. **[PENDING: state whether the averaging is applied to logits or to probabilities, and use the same choice in every chapter and slide. Averaging logits and applying the sigmoid once is the simpler option, but the team must decide.]**

Set Transformer aggregation. The pair $\{\mathbf{z}_A, \mathbf{z}_B\}$ is treated as a set and passed through a Set Attention Block (SAB), in which each embedding attends to both embeddings in the set (itself included):

$$\text{SAB}(X) = \text{MultiHead}(X, X, X), \quad (4.3)$$

where queries, keys, and values all come from the same set X . This lets the representation of each molecule be adjusted using the other before the two are combined. The attention mechanism follows the definition in Chapter 3 **[TODO: \ref]**. The adjusted pair is then reduced to a single blend embedding $\mathbf{z}_{\text{blend}} \in \mathbb{R}^d$ by Pooling by Multihead Attention (PMA), which uses learned attention weights instead of a plain average. **[PENDING: number of SAB blocks, number of attention heads, hidden size, and whether PMA uses one seed vector; values from the earlier notebook (2 blocks, 4 heads) are not inherited automatically.]** Both SAB and PMA are permutation-invariant in the sense needed here: swapping $\mathbf{z}_A$ and $\mathbf{z}_B$ does not change $\mathbf{z}_{\text{blend}}$. This module is a learned way of mixing two embeddings. This study does not claim that it models physical or receptor-level interaction between the molecules.

Complexity. Once the models are implemented, the study will report the number of trainable parameters of each full model and a measure of computation cost (for example, training time per epoch and inference time per batch on the same hardware), so that any accuracy difference can be read together with the cost of obtaining it. **[FILL IN: parameter counts and timings.]**

4.4.3 Output Head and Loss

The blend embedding is passed through a Multi-Layer Perceptron that maps $d \rightarrow h \rightarrow L$, where h is a hidden width and L is the final target vocabulary size (Section 4.2.3). **[PENDING: h and L ; the earlier notebook used $h = 128$.]** For order-averaged concatenation the first layer takes $2d$ inputs. The output layer applies an independent sigmoid to each of the L nodes, because odor descriptors are multi-label: each descriptor is its own present/absent decision, unlike softmax, which forces the outputs to sum to one. The framework for multi-label learning and the Asymmetric Focal Loss are defined in Chapter 3 **[TODO: \ref]**. Following Hatami and Moradi [19], the loss uses a focusing parameter of $\gamma_p = 0$ for positives and $\gamma_n = 2$ for negatives **[PENDING: confirm these values for the revised study]**. The per-cell loss ℓ_{ij} of this function enters the masked loss of Section 4.3. All three variants use the same loss, optimizer, learning-rate schedule, early-stopping rule, and training budget.

4.5 Component-Disjoint Partitioning and Graph Carving

This section describes how the data are divided into training, validation, and test sets. The aim is that no molecule that appears in a training blend also appears in a test blend. This section describes the procedure; it does not claim that the test set contains only chemically unfamiliar molecules.

Why random pair splitting leaks components. A blend dataset is a collection of pairs, and the same molecule appears in many pairs. If pairs are split at random, a molecule seen in a training pair is very likely to appear again in a test pair, so test performance partly measures memorization of molecules rather than prediction for new ones. Group-aware splitting is the standard remedy for this kind of overlap [24] (see Section 2.13).

4.5.1 Scaffold Split Decay

Scaffold splitting, which groups molecules by shared core framework, is often used for this purpose. In sparse chemical spaces, however, most scaffold groups contain very few molecules, and scaffold splitting then behaves much like random splitting (Section 2.13). To check this on the present data,

the molecule-to-scaffold ratio will be computed:

$$R = \frac{N_{\text{molecules}}}{N_{\text{scaffolds}}} \quad (4.4)$$

where $N_{\text{molecules}}$ is the number of unique canonical molecules and $N_{\text{scaffolds}}$ is the number of distinct Bemis–Murcko frameworks produced by RDKit. A value of R near 1 means almost every scaffold holds a single molecule. R will be computed on the merged master dictionary and compared with the collapse threshold $R < 2.0$ reported by Fooladi et al. [13]. **[CONFIRM: check this citation key against references.bib.]**

4.5.2 Cluster-Based Partitions

Molecules will be grouped by structural similarity and whole groups will be assigned to one partition. The planned pipeline is: compute a binary structural fingerprint for every unique molecule; reduce it to a low-dimensional map with UMAP; and cluster the map with K-Means into $k = 7$ clusters. The number of clusters is fixed at seven, following Guo et al. [16], who found seven UMAP clusters to outperform alternative clustering methods on molecular data. A candidate fingerprint is ECFP4 (2048 bits, radius 2).

[PENDING: fingerprint type and size, UMAP settings, random seeds, and the cluster-to-partition rule are not final. They will be fixed and written down before any final evaluation, and the values of the earlier notebook will be reused only if the team deliberately re-approves them. The value $k = 7$ is the only clustering setting treated as fixed.]

The rule that assigns clusters to training, validation, and test partitions will be *declared in advance and will not depend on any label*. For example, it may depend only on cluster sizes (such as assigning clusters to partitions so that the molecule counts approximate a target ratio). **[PENDING: state the exact rule.]** The selection will not look at which descriptors occur in which cluster, and it will not look at test labels. Once the partitions are fixed they are not changed; the number of molecules, pairs, and descriptor support in each partition are reported afterwards as outcomes, not used as selection criteria. If any inspection of test labels turns out to be unavoidable, it will be disclosed, and the test set will not be described as untouched.

4.5.3 The Graph-Carving Protocol

The blend dataset is treated as a graph in which each node is a unique canonical molecule and each edge is a documented blend between two molecules. Every molecule is assigned to a partition by its cluster. Carving then examines each edge: an edge (blend) is *retained only if both molecules belong to the same assigned partition*, and discarded otherwise. After carving, no molecule is shared between any two partitions, so no molecule component can leak from training into validation or test. Sisson et al. [44] applied a similar carving to a random 50:50 molecule partition, which discarded about 83,000 pairs and left about 44,000 training and 40,000 testing pairs. This study uses clusters instead of a random molecule split, so its pair counts will differ and are reported only after the procedure is run. **[FILL IN: pairs retained and discarded per partition.]**

The validation set is used for early stopping, hyperparameter decisions, and selection of decision thresholds. The test set is kept sealed and used only in the final evaluation.

Component-disjoint partitioning reduces component and structural overlap between partitions. It does not guarantee that test molecules belong to unfamiliar chemical families, because clusters that are near each other on the map can still contain similar molecules.

4.5.4 Training-Only Vocabulary Filter

Carving changes how often each descriptor occurs, so the final target vocabulary is chosen after partitioning, from training data only. A standardized descriptor d is kept if it has enough positive examples to learn from:

$$d \in \mathcal{D}_{\text{target}} \iff c_{\text{train}}(d) \geq x \quad (4.5)$$

where $c_{\text{train}}(d)$ is the number of training blends in which d is recorded as positive, and x is a minimum-support threshold. **[PENDING: the value of x ; the earlier notebook used $x = 5$ on both training and test sides, and the test-side condition is removed here.]** The size of $\mathcal{D}_{\text{target}}$ is L . In the Gold-only condition, $c_{\text{train}}(d)$ counts eligible positives. After the vocabulary is locked, the number of validation and test positives and negatives for each descriptor is *reported* but is not used to add or remove descriptors. Descriptors that end up with too little test support are handled by the insufficient-support rule of Section 4.3 and are listed in the report.

4.6 Experiments

All experiments hold the partitions, the encoder initialization policy, the loss, the optimizer, the stopping rule, the training budget, and the metric suite constant across the three aggregation methods. They will show whether attention-based aggregation performs better than simpler alternatives; a finding that a simpler baseline performs as well or better is a valid and informative result and is reported as such. Each configuration is run with the same set of random seeds for every aggregation method. **[PENDING: the number of seeds.]**

4.6.1 Experiments A and B: Main Comparison (RQ1)

Experiment A trains each model in the full-label condition, using all cells. Experiment B trains each model in the Gold-only condition, using masked loss over eligible cells (Section 4.3). Both are evaluated on the held-out test partition. Experiment A is also scored on all cells, and Experiment B is scored on eligible cells with the same mask. Reporting both makes visible how much of any difference depends on which cells are counted.

4.6.2 Experiment C: Controlled Label Corruption (RQ3)

To examine how the comparison changes as training labels become less reliable, labels will be corrupted on purpose, in the training partition only. Validation and test labels are never changed.

For a noise level $p \in \{0, 0.10, 0.20, 0.30\}$, a fraction p of the eligible positive cells in the training labels is flipped to absent, and an equal number of eligible negative cells is flipped to present, which keeps the density of positives unchanged. Only eligible cells can be flipped, so the same mask logic as Section 4.3 applies. For each combination of noise level and random seed, one corruption mask is generated and *reused across all three aggregation methods*; a new mask is generated for every seed and noise level. The run at $p = 0$ is the uncorrupted Gold-only run of Experiment B and is reused. Models are retrained at each level and scored on the unchanged validation and test labels. The change in macro F1 from $p = 0$ to $p = 0.30$ is then compared across methods using the paired bootstrap of Section 4.8.

Random flipping tests robustness to label errors in general. It does not reproduce every real process by which catalog errors arise.

4.6.3 Blend-Type Analysis (RQ2)

To see whether the relative performance of the methods differs by catalog-label pattern, each test blend is compared with the recorded descriptors of its two parent molecules. The terms below describe patterns in the catalog records; they are not claims about what a person would perceive.

- **Catalog-label loss:** a descriptor recorded for either parent is not recorded for the blend.
- **Catalog-label gain:** the blend has a recorded descriptor that is not recorded for either parent.
- **Additive:** the blend shows neither loss nor gain.

Descriptive statistics of the label matrix (label density, common co-occurrences, and the rate at which a blend’s labels are additive with respect to its parents) are reported for the dataset as a whole. They are used to describe the data and to support this subgroup analysis; they are not used to filter data. The sizes of these three groups are reported and the model comparison is repeated within each group. Because these groups overlap in how they are defined and the analysis is split into subgroups, it is treated as a secondary subgroup and error analysis rather than a confirmatory test. If any group contains fewer than 5% or more than 95% of the test blends, blends are instead divided by the overlap between the blend descriptors and the union of the parents’ descriptors.

4.7 Evaluation Metrics

This section defines the metrics and explains why each is needed when most label cells are negative.

4.7.1 Multi-Label Metrics

Let TP be a descriptor correctly predicted present, FP a descriptor incorrectly predicted present, FN a present descriptor that was missed, and TN a descriptor correctly predicted absent. Most cells in the label matrix are negative, so a model can look accurate by predicting “absent” every time. Precision, recall, and F_1 use only TP, FP, and FN [37], so they ignore the many easy true negatives, and a model that predicts nothing receives $F_1 = 0$.

- **Macro F1 (primary metric).** F_1 is computed for each descriptor separately and then averaged with equal weight, so rare descriptors count as much as common ones [37]. This choice

is declared before experimentation and must be the same in Chapters 1–4. [**CONFIRM: Chapter 1.**]

- **Micro F1.** TP, FP, and FN are pooled across all descriptors before F_1 is computed; it summarizes overall performance and favors common descriptors.
- **PR-AUC.** The area under the precision–recall curve, which measures how well the model ranks true descriptors above false ones across all thresholds. It is reported both macro-averaged over descriptors (primary version) and micro-averaged. [**PENDING: confirm that both versions are reported.**]
- **Hamming loss.** The fraction of evaluated cells that are predicted incorrectly (below).

Thresholds. Each descriptor’s probability is turned into a present/absent prediction with a decision threshold. Thresholds are chosen on validation data only and then applied unchanged to the test data. Threshold selection matters a good deal for imbalanced multi-label problems [54]. [**CONFIRM: check citation key; PENDING: whether one global threshold or a per-descriptor threshold is used.**]

4.7.2 Masked Metrics

All metrics are computed only over cells with $M_{ij} = 1$, using the mask of the condition being evaluated (all ones in the full-label condition). For macro metrics, descriptors with insufficient eligible support are excluded and listed (Section 4.3). With a mask, Hamming loss is

$$\text{HL}_M = \frac{\sum_{i=1}^N \sum_{j=1}^L M_{ij} \mathbf{1}(y_{ij} \neq \hat{y}_{ij})}{\sum_{i=1}^N \sum_{j=1}^L M_{ij}} \quad (4.6)$$

where N is the number of evaluated blends, L is the number of target descriptors, y_{ij} is the label, $\hat{y}_{ij} \in \{0, 1\}$ is the thresholded prediction, and $\mathbf{1}(\cdot)$ equals 1 if its argument is true and 0 otherwise. Smaller values are better, with $\text{HL}_M = 0$ for a perfect match. In the full-label condition $M_{ij} = 1$ everywhere and this is the ordinary Hamming loss.

All-negative reference. Hamming loss also rewards the many easy negatives, so it cannot be read alone. The *evaluated-cell density* is

$$D_M = \frac{\sum_i \sum_j M_{ij} y_{ij}}{\sum_i \sum_j M_{ij}} \quad (4.7)$$

the fraction of evaluated cells that are positive. A model that predicts every cell absent has $HL_M = D_M$, which serves as the all-negative reference. It is computed separately for each partition (D_{train} , D_{val} , D_{test}) and for each mask, and a value from one setting is never reused in another. **[FILL IN: densities and the all-negative reference, after the pipeline is run.]** Every model’s Hamming loss is reported next to this reference.

4.8 Statistical Comparison

Differences between models could reflect chance in which blends happen to be in the test set, or chance in training. These two sources are handled separately.

Variability across seeds. Optimization variability comes from random initialization and data order. Each model is trained with the same set of random seeds, and results are summarized across seeds (mean and spread) separately from the bootstrap below.

Bootstrap over blends. Descriptor outcomes within one blend are correlated, so individual cells are not independent. The test blends are therefore resampled as whole rows: a sampled blend keeps all of its descriptor cells together, with its mask. Naive paired t -tests are not appropriate for comparing machine-learning models, because they are sensitive to outliers and not valid when resampled test sets are used [37], and bootstrap resampling shows both whether a difference exists and how large it is [5].

From the N test blends, N rows are drawn with replacement, B times **[PENDING: the number of replicates B will be fixed before final evaluation]**. All models being compared are scored on the *same* resampled indices, giving a paired comparison. For replicate b the paired difference is

$$\Delta(b) = \text{Metric}_{\text{model 1}}(b) - \text{Metric}_{\text{model 2}}(b). \quad (4.8)$$

The direction of a difference is stated with each result: higher is better for macro and micro F1 and for PR-AUC, and lower is better for Hamming loss, so a positive Δ favors model 1 for the first three and model 2 for Hamming loss. The comparison is repeated for each metric and for each pair of aggregation methods (three pairs). **[PENDING: whether a correction for multiple comparisons is applied, or whether the primary comparison is limited to macro F1.]**

Intervals and effect size. The significance level is $\alpha = 0.05$. A 95% percentile confidence interval is taken from the 2.5th and 97.5th percentiles of the B differences, and an interval that excludes zero is read as a difference at $\alpha = 0.05$. Because statistical significance does not show whether a difference matters in practice, the effect size is reported as the mean difference with its 95% interval, in the units of each metric. For Experiment C, the bootstrap is applied to the change in macro F1 between noise levels.

4.9 Reporting Conventions

Experiments that have not been run are described in the future tense. Any pipeline demonstration run before the final protocol is fixed will be labelled preliminary and will not be reported as a result. Output from the earlier notebook is not reported as a result of the revised study. The study does not assume that the Set Transformer will outperform the baselines; if mean pooling or order-averaged concatenation matches or exceeds it, that outcome will be reported and discussed on equal footing.

Appendix A

Chemical Feature Vectors

The exact composition of the 46-dimensional node feature vector and the 10-dimensional edge feature vector used for the intra-molecular GAT encoder are detailed below.

Chemical Property	Dimensions	Mathematical / One-Hot Representation & Details
Atomic Identity	9	One-hot encoding of key organic and halogen elements in flavor/fragrance compounds: [C, N, O, S, F, Cl, Br, I, Other].
Formal Charge	1	Scalar representation of the localized net electrical charge on the atom.
Atomic Degree	6	One-hot representation of the number of connected covalent bonds to heavy atoms: [0, 1, 2, 3, 4, 5+].
Explicit Valence	7	One-hot encoding of the total valence electrons committed to bonding in the molecule: [0, 1, 2, 3, 4, 5, 6+].
Hybridization State	6	One-hot representation of the orbital configuration: [sp , sp^2 , sp^3 , sp^3d , sp^3d^2 , Other].
Aromaticity	1	Binary flag indicating if the atom is part of a resonance-stabilized aromatic ring structure.
Attached Hydrogens	5	One-hot encoding of the number of hydrogens bound to this heavy atom: [0, 1, 2, 3, 4+].
Ring Size Participation	6	Binary flags representing specific ring sizes to map cyclic functional groups: [3-ring, 4-ring, 5-ring, 6-ring, 7-ring, 8-ring].
Ring Membership	1	Binary flag indicating whether the atom is enclosed within any ring structure.
Chirality Tag	4	One-hot representation of tetrahedral stereocenter configuration: [Unspecified, Clockwise (CW), Counterclockwise (CCW), Other].

Table A.1: Specification of the 46-Dimensional Atom (Node) Feature Vector (X_i).

Chemical Property	Dimensions	Mathematical / One-Hot Representation & Details
Bond Type	4	One-hot representation of the bond's electronic sharing structure: [Single, Double, Triple, Aromatic].
Conjugation	1	Binary flag indicating if the bond belongs to a continuous, overlapping pi-electron system.
Ring Membership	1	Binary flag indicating if the bond is closed within a ring structure.
Stereochemical Config	4	One-hot representation of spatial configuration around double bonds: [None, Stereo-Z, Stereo-E, Other].

Table A.2: Specification of the 10-Dimensional Bond (Edge) Feature Vector (E_{ij}).

A.1 Symmetry Collapse: Representation-Theoretic Derivation

This appendix provides the full representation-theoretic argument for the expressivity limitation of linear, permutation-invariant readouts at the mixture level, summarised in Section 2 of the main text.

Consider a molecular graph G whose set of node embeddings M occupies a d -dimensional representation space $\mathcal{U}$. The structural symmetries of the graph are captured by its automorphism group $\mathcal{H}$, which collects all permutations of the node indices that leave the graph structure unchanged. When a linear, $\mathcal{H}$ -invariant readout function f is applied to aggregate these embeddings, representation theory requires that it factor through the group-averaging Reynolds operator p_{avg} , defined as [49]:

$$p_{\text{avg}} = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} \rho(h). \quad (\text{A.1})$$

The Reynolds operator projects the full representation space $\mathcal{U}$ onto its trivial, symmetry-fixed subspace $\mathcal{U}_1$. As a consequence, any linear $\mathcal{H}$ -invariant map decomposes into a downstream linear map composed with this group-averaging projection [49]:

$$f = f' \circ p_{\text{avg}}. \quad (\text{A.2})$$

This factorization establishes that a linear readout can only operate on the information that survives in $\mathcal{U}_1$. The non-trivial, symmetry-sensitive components $\mathcal{U}_2 \oplus \dots \oplus \mathcal{U}_m$, which encode the node-to-node variation responsible for structural differences between molecules, are irreversibly discarded by the projection [49]. As a result, even when the Level 1 GAT encoder produces embeddings that are genuinely sensitive to structural differences between molecules, that information does not survive a linear mixture-level readout. Two distinct molecular combinations that share the same element-wise sum or mean are mapped to identical blend vectors and become indistinguishable to any downstream classifier, regardless of how expressive the encoder is. This expressivity ceiling motivates replacing linear pooling at the mixture level with an attention-based readout that operates on the full representation space [44, 49].

A.2 Dataset Profiles and Canonical Vocabulary

A.2.1 Single-Molecule Harmonized Dataset Sample

Table A.3 provides representative single-molecule entries resulting from the consolidation of the GoodScents and Leffingwell databases under RDKit organic sanitization and canonicalization.

Table A.3: Representative sample of harmonized single odorants.

CID	Common Name	Canonical SMILES	Harmonized Descriptors
1183	Vanillin	<chem>COC1CC(C=O)CCC1O</chem>	chocolate, creamy, sweet, vanilla
1254	Menthol	<chem>CC1CCC(C(C)C)C(O)C1</chem>	cooling, mentholic, minty, woody
3314	Eugenol	<chem>C=CCc1ccc(O)c(OC)c1</chem>	cinnamon, clove, phenolic, spicy, woody
10582	Myrtenol	<chem>CC1(C)C2CC=C(CO)C1C2</chem>	balsamic, camphoreous, herbal, pine
31276	Isoamyl acetate	<chem>CC(=O)OCCC(C)C</chem>	banana, estery, fruity, pear, sweet
87630	2-Decen-1-ol	<chem>CCCCCCCC=CCO</chem>	citrus, fatty, floral, rose, waxy
637511	Cinnamaldehyde	<chem>O=C/C=C/c1ccccc1</chem>	candy, cinnamon, spicy, warm

A.2.2 Pairwise Blend Mixture Sample

Table A.4 presents sample binary mixtures from the pairwise blend benchmark dataset compiled by Sisson et al. [44], illustrating the constituent molecules and their emergent olfactory descriptors.

Table A.4: Sample pairwise odor mixtures from the Sisson et al. benchmark with emergent descriptors.

Blend	Component 1 (CID)	Component 2 (CID)	Emergent Notes
B-01	Acetaldehyde (177)	2-Heptanol (10976)	citrus, fruity, pungent
B-02	Acetoin (179)	Bornylcyclohexanol (111394)	buttery, creamy, woody
B-03	2-Oxobutanoic acid (58)	Methyl dihydrojasmonate (102861)	brown, caramellic, floral
B-04	2-Oxobutanoic acid (58)	γ -Undecalactone (7714)	brown, caramellic, fruity
B-05	Pentopyranose (229)	Perillaldehyde (16441)	herbal, spicy
B-06	2-Oxobutanoic acid (58)	Butyric acid (264)	cheesy, sour
B-07	Cinnamaldehyde (637511)	Eugenol (3314)	spicy

A.2.3 Standardized 104-Odor Descriptor Vocabulary Gate

Table A.5 enumerates the complete 104-descriptor standardized vocabulary gate enforced during Phase 1 preprocessing.

Table A.5: The 104 standardized odor descriptors (alphabetical).

1. acetic	27. clean	53. honey	79. potato
2. acidic	28. cocoa	54. jammy	80. powdery
3. alcoholic	29. coconut	55. juicy	81. pungent
4. aldehydic	30. coffee	56. lactonic	82. roasted
5. alliaceous	31. cooling	57. leathery	83. rooty
6. amber	32. corn	58. licorice	84. rummy
7. ammoniacal	33. coumarinic	59. malty	85. salty
8. animal	34. creamy	60. marine	86. smoky
9. anise	35. dairy	61. meaty	87. soapy
10. aromatic	36. dusty	62. medicinal	88. solvent
11. balsamic	37. earthy	63. melon	89. sour
12. berry	38. eggy	64. mentholic	90. spicy
13. bitter	39. estery	65. minty	91. sulfurous
14. bready	40. ethereal	66. moldy	92. sweet
15. brown	41. fatty	67. mossy	93. tarragon
16. burnt	42. fermented	68. mushroom	94. thujonic
17. buttery	43. fishy	69. musk	95. toasted
18. cabbage	44. floral	70. mustard	96. tobacco
19. camphoreous	45. fresh	71. musty	97. tomato
20. caramellic	46. fruity	72. nutty	98. tonka
21. celery	47. fungal	73. oily	99. tropical
22. cheesy	48. fusel	74. onion	100. vanilla
23. chemical	49. garlic	75. orris	101. vegetable
24. cherry	50. green	76. peach	102. waxy
25. chocolate	51. hay	77. phenolic	103. winy
26. citrus	52. herbal	78. pine	104. woody

REFERENCES

- [1] R. ACHEBOUCHE, A. TROMELIN, K. AUDOUZE, AND O. TABOUREAU, *Application of artificial intelligence to decode the relationships between smell, olfactory receptors and small molecules*, Scientific Reports, 12 (2022), p. 18817.
- [2] R. L. ALMEIDA, V. G. MALTAROLLO, AND F. G. F. COELHO, *Overcoming class imbalance in drug discovery problems: Graph neural networks and balancing approaches*, Journal of Molecular Graphics and Modelling, 126 (2024), p. 108627.
- [3] D. AMETA, S. KUMAR, R. MISHRA, L. BEHERA, A. CHAKRABORTY, AND T. SANDHAN, *Odor classification: Exploring feature performance and imbalanced data learning techniques*, PLOS ONE, 20 (2025), p. e0322514.
- [4] A. R. N. AOUICHAOU, F. FAN, S. S. MANSOURI, J. ABILDSKOV, AND G. SIN, *Combining group-contribution concept and graph neural networks toward interpretable molecular property models*, Journal of Chemical Information and Modeling, 63 (2023), pp. 725–744.
- [5] Y. BESTGEN, *Please, don't forget the difference and the confidence interval when seeking for the state-of-the-art status*, in Proceedings of the Thirteenth Language Resources and Evaluation Conference, Marseille, France, June 2022, European Language Resources Association, pp. 5956–5962.
- [6] A. L. BIERLING, A. CROY, T. JESGARZEWSKY, M. ROMMEL, G. CUNIBERTI, T. HUMMEL, AND I. CROY, *A dataset of laymen olfactory perception for 74 mono-molecular odors*, Scientific Data, 12 (2025), p. 347.
- [7] J. B. CASTRO, T. J. GOULD, R. PELLEGRINO, Z. LIANG, L. A. COLEMAN, F. PATEL, D. S. WALLACE, T. BHATNAGAR, J. D. MAINLAND, AND R. C. GERKIN, *Pyrfume: A window to the world's olfactory data*, bioRxiv, (2022).
- [8] C. CHEN, M. SHENG, H. YUAN, X. PAN, H. TIAN, AND X. LOU, *Perceptual interaction of aroma compounds and their effects on the enhancement of flavor quality in foods: a review*, Trends in Food Science & Technology, 165 (2025), p. 105315.
- [9] S. CHITHRANANDA, J. AMORES, AND K. K. YANG, *Mapping the combinatorial coding between olfactory receptors and perception with deep learning*, bioRxiv, (2026).
- [10] G. COUREAUD, T. THOMAS-DANGUIN, J.-C. SANDOZ, AND D. A. WILSON, *Biological constraints on configural odour mixture perception*, J Exp Biol, 225 (2022).

- [11] R. DATTA, J. NISTANE, A. SOSE, H. SAHU, AND R. RAMPRASAD, *Machine learning for green solvents: Assessment, selection and substitution*, Advanced Science, (2025).
- [12] M. FEY AND J. E. LENSSEN, *Fast graph representation learning with pytorch geometric*, CoRR, abs/1903.02428 (2019).
- [13] H. FOOLADI, T. N. L. VU, M. MATHEA, AND J. KIRCHMAIR, *Evaluating machine learning models for molecular property prediction: Performance and robustness on out-of-distribution data*, Journal of Chemical Information and Modeling, 65 (2025), pp. 9871–9891.
- [14] Y. FUKUTANI, M. ABE, H. SAITO, R. EGUCHI, T. TAZAWA, C. A. DE MARCH, M. YOHDA, AND H. MATSUNAMI, *Antagonistic interactions between odorants alter human odor perception*, Current Biology, 33 (2023), pp. 2235–2245.e4. doi: 10.1016/j.cub.2023.04.072.
- [15] D. GASSER, S. SINGH, J. FREIHERR, AND A. GRASSKAMP, *Owsum: algorithmic odor prediction and insight into structure-odor relationships*, Journal of Cheminformatics, 15 (2023).
- [16] Q. GUO, S. HERNANDEZ-HERNANDEZ, AND P. J. BALLESTER, *Umap-based clustering split for rigorous evaluation of ai models for virtual screening on cancer cell lines**, Journal of Cheminformatics, 17 (2025), p. 94.
- [17] R. HAGAN, D. MOORE, F. MCGLONE, AND S. C. WALKER, *Olfactory scene analysis: Does analytical visual processing predict superior identification of component odours in a complex mixture?*, Perception, 54 (2025), pp. 408–430. PMID: 40165414.
- [18] S. HAN, H. FU, Y. WU, G. ZHAO, Z. SONG, F. HUANG, Z. ZHANG, S. LIU, AND W. ZHANG, *Himgnn: a novel hierarchical molecular graph representation learning framework for property prediction*, Briefings in Bioinformatics, 24 (2023), p. bbad305.
- [19] F. HATAMI AND M. MORADI, *Asymmetric focal loss improves graph neural network prediction of drug-drug interactions*, 2026.
- [20] Y. P. HENG, H. Y. LEE, J. W. CHONG, R. R. TAN, K. B. AVISO, AND N. G. CHEM-MANGATTUVALAPPIL, *Incorporating machine learning in computer-aided molecular design for fragrance molecules*, Processes, 10 (2022).
- [21] M. HLADIŠ, M. LALIS, S. FIORUCCI, AND J. TOPIN, *Matching receptor to odorant with protein language and graph neural networks*, in The Eleventh International Conference on Learning Representations, 2023.
- [22] B. HU, H. ZHENG, H. LIANG, H. LV, Q. XIAO, Y. JIANG, Q. LU, Y. NIU, H. WANG, AND Z. XIAO, *Elucidating odorant masking effects in food matrices through olfactory receptor-based profiling*, bioRxiv, (2026).
- [23] J. HUANG, J. LIN, R. YUENG, S. WU, L. SOLLA, AND T. ACREE, *Masking effects on isovaleric acid recognition by sub-threshold odor mixture*, Chemical Senses, 48 (2023), p. bjad047.
- [24] R. JOERES, D. B. BLUMENTHAL, AND O. V. KALININA, *Data splitting to avoid information leakage with datasail*, Nature Communications, 16 (2025), p. 3337.

- [25] A. KENGKANNA AND M. OHUE, *Enhancing property and activity prediction and interpretation using multiple molecular graph representations with mmgx*, Communications Chemistry, 7 (2024), p. 74.
- [26] M. LALIS, M. HLADIŠ, S. A. KHALIL, L. BRIAND, S. FIORUCCI, AND J. TOPIN, *M2or: a database of olfactory receptor-odorant pairs for understanding the molecular mechanisms of olfaction*, Nucleic Acids Research, 52 (2024), pp. D1370–D1379.
- [27] B. K. LEE, E. J. MAYHEW, B. SANCHEZ-LENGELING, J. N. WEI, W. W. QIAN, K. A. LITTLE, M. ANDRES, B. B. NGUYEN, T. MOLOY, J. YASONIK, J. K. PARKER, R. C. GERKIN, J. D. MAINLAND, AND A. B. WILTSCHKO, *A principal odor map unifies diverse tasks in olfactory perception*, Science, 381 (2023), pp. 999–1006.
- [28] J. LEE, Y. LEE, J. KIM, A. R. KOSIOREK, S. CHOI, AND Y. W. TEH, *Set transformer: A framework for attention-based permutation-invariant neural networks*, 2019.
- [29] Q. LI, S. LI, J. LUO, AND H. YUAN, *Predictive models of odor contribution and thresholds for volatiles in identification of novel crop aroma compounds*, Metabolites, 15 (2025).
- [30] S. LI, X. XIA, S. GE, AND T. LIU, *Selection-based pseudo-labeling for multi-label learning with noisy labels: Estimating the noise transition*, in Advances in Neural Information Processing Systems, 2022.
- [31] K. LÓPEZ-PÉREZ, J. F. AVELLANEDA-TAMAYO, L. CHEN, E. LÓPEZ-LÓPEZ, K. E. JUÁREZ-MERCADO, J. L. MEDINA-FRANCO, AND R. A. MIRANDA-QUINTANA, *Molecular similarity: Theory, applications, and perspectives*, Artificial Intelligence Chemistry, 2 (2024), p. 100077.
- [32] E. MAKRI, N. NAKIS, L. SISSON, G. MINSKY, L. TASSIULAS, V. SATARIFARD, AND N. A. CHRISTAKIS, *Benchmark for assessing olfactory perception of large language models*, 2026.
- [33] C. PATINHA CALDEIRA, R. FARCAL, C. MORETTI, L. MANCINI, H. RAUSCHER, K. RASMUSSEN, J. RIEGO SINTES, AND S. SALA, *Safe and sustainable by design chemicals and materials: Review of safety and sustainability dimensions, aspects, methods, indicators, and tools*, EUR 30991 EN, Joint Research Centre (JRC), European Commission, Luxembourg, 2022.
- [34] A. PAUL, *Dextrosinistral reading of smiles notation: Investigation into origin of non-sense code from string manipulations*, Digital Chemical Engineering, 15 (2025), p. 100222.
- [35] R. PELLEGRINO, E. J. MAYHEW, J. MARGOLIS, M. ANDRES, A. B. WILTSCHKO, R. C. GERKIN, AND J. D. MAINLAND, *Odors smell like their components: A linear framework for predicting olfactory mixture perception*, bioRxiv, (2026).
- [36] S. QIN, S. JIANG, J. LI, P. BALAPRAKASH, R. C. VAN LEHN, AND V. M. ZAVALA, *Capturing molecular interactions in graph neural networks: a case study in multi-component phase equilibrium*, Digital Discovery, 2 (2023), pp. 138–151.
- [37] O. RAINIO, J. TEUHO, AND R. KLÉN, *Evaluation metrics and statistical tests for machine learning*, Scientific Reports, 14 (2024), p. 6086.
- [38] B. C. L. RODRIGUES, V. V. SANTANA, S. MURINS, AND I. B. R. NOGUEIRA, *Molecule generation and optimization for efficient fragrance creation*, Industrial & Engineering Chemistry Research, 63 (2024), pp. 14480–14494.

- [39] S. ROMAGNY, G. COUREAUD, AND T. THOMAS-DANGUIN, *Developmental switch from elemental to configural processing of odor mixtures in humans*, *Developmental Psychobiology*, 68 (2026), p. e70188.
- [40] K. SAINI AND V. RAMANATHAN, *Predicting odor from molecular structure: a multi-label classification approach*, *Scientific Reports*, 12 (2022), p. 13863.
- [41] A. SAJAN, S. SLUIS, R. HAYDARLOU, S. ABELN, P. LISENA, R. TRONCY, C. VERBEEK, I. LEEMANS, AND H. MOUHIB, *Hierarchies of smell: Structuring the molecular odor space using semantic taxonomies and machine learning*, *Chemical Senses*, 51 (2026), p. bjag020.
- [42] E. I. SANCHEZ MEDINA, S. LINKE, M. STOLL, AND K. SUNDMACHER, *Gibbs–helmholtz graph neural network: capturing the temperature dependency of activity coefficients at infinite dilution*, *Digital Discovery*, 2 (2023), pp. 781–798.
- [43] H. SHIMAKAWA, A. KUMADA, AND M. SATO, *Extrapolative prediction of small-data molecular property using quantum mechanics-assisted machine learning*, *npj Computational Materials*, 10 (2024), p. 11.
- [44] L. SISSON, A. A. BARSAINYAN, M. SHARMA, AND R. KUMAR, *Deep learning for odor prediction on aroma-chemical blends*, *ACS Omega*, 10 (2025), pp. 8980–8992.
- [45] L. J. S. SOBRAL, F. L. FEITOSA, J. A. M. KUNZ, P. K. DINIZ, J. F. AVELLANEDA-TAMAYO, J. L. MEDINA-FRANCO, A. G. FILHO, AND C. H. ANDRADE, *Odor-sight: An interpretable graph neural network web platform for prediction of odorant activity*, *Journal of Chemical Information and Modeling*, 66 (2026), pp. 9721–9728.
- [46] H. SONG, M. KIM, D. PARK, Y. SHIN, AND J.-G. LEE, *Learning from noisy labels with deep neural networks: A survey*, *IEEE Transactions on Neural Networks and Learning Systems*, 34 (2023), pp. 8135–8153.
- [47] S. STESHIN, *Lo-hi: Practical ml drug discovery benchmark*, 2023.
- [48] J. SUH, Y. HONG, AND C. PARK, *A comparative study of machine learning models on molecular fingerprints for odor decoding*, *Communications Chemistry*, 8 (2025), p. 278.
- [49] M. TALHI, A. WOLF, AND A. MONOD, *Breaking symmetry bottlenecks in gnn readouts*, 2026.
- [50] A. N. TAREKEGN, M. GIACOBINI, AND K. MICHALAK, *A review of methods for imbalanced multi-label classification*, *Pattern Recognition*, 118 (2021), p. 107965.
- [51] G. TOM, C. T. SER, E. RAJAONSON, S. LO, H. S. PARK, B. LEE, AND B. SANCHEZ-LENGELING, *Does this smell the same? learning representations of olfactory mixtures using inductive biases*, *Machine Learning: Science and Technology*, 6 (2025).
- [52] A. VANACORE AND M. S. PELLEGRINO, *Benchmarking procedures for characterizing the extent of rater agreement: A comparative study*, *Quality and Reliability Engineering International*, 38 (2022), pp. 1404–1415.
- [53] J. WANG, Q. ZHANG, W. FAN, Q. SHI, J. MAO, J. XIE, G. CHAI, AND C. ZHANG, *Deciphering olfactory receptor binding mechanisms: a structural and dynamic perspective on olfactory receptors*, *Frontiers in Molecular Biosciences*, Volume 11 - 2024 (2025).

- [54] T. WEN, X. CAI, AND J. LI, *Graph neural networks vs. traditional qsar: A comprehensive comparison for multi-label molecular odor prediction*, *Molecules*, 30 (2025).
- [55] D. S. WIGH, J. M. GOODMAN, AND A. A. LAPKIN, *A review of molecular representation in the age of machine learning*, *WIREs Computational Molecular Science*, 12 (2022), p. e1603.
- [56] Y.-X. WU, X. WANG, A. ZHANG, X. HE, AND T.-S. CHUA, *Discovering invariant rationales for graph neural networks*, 2022.
- [57] Z. WU, J. WANG, H. DU, D. JIANG, Y. KANG, D. LI, P. PAN, Y. DENG, D. CAO, C.-Y. HSIEH, AND T. HOU, *Chemistry-intuitive explanation of graph neural networks for molecular property prediction with substructure masking*, *Nature Communications*, 14 (2023), p. 2585.
- [58] H. XIE, J. SUN, Y. SHAO, S. LI, S. HOU, Y. SUN, AND J. WANG, *Molecular odor prediction based on multi-feature graph attention networks*, 2025.
- [59] L. XU, D.-J. ZOU, AND S. FIRESTEIN, *Odor mixtures: A chord with silent notes*, *Frontiers in Ecology and Evolution*, Volume 11 - 2023 (2023).
- [60] M. ZAHEER, S. KOTTUR, S. RAVANBAKHS, B. POCZOS, R. SALAKHUTDINOV, AND A. SMOLA, *Deep sets*, 2018.
- [61] M. ZHANG, Y. HIKI, A. FUNAHASHI, AND T. J. KOBAYASHI, *A deep position-encoding model for predicting olfactory perception from molecular structures and electrostatics*, *NPJ systems biology and applications*, 10 (2024), p. 76.
- [62] W. ZHU, Y. ZHANG, D. ZHAO, J. XU, AND L. WANG, *Hignn: Hierarchical informative graph neural networks for molecular property prediction equipped with feature-wise attention*, 2022.